Mitigazione delle vulnerabilità del DNS: Analisi dell'attacco di Cache Poisoning e implementazione di DNSSEC

Studente:

Pasquale Paolo Silvenni (M63001717)

Introduzione
Capitolo 1: Progettazione e Setup dell'Infrastruttura
- 1.1 Architettura della Rete
- 1.2 Dimensionamento delle Risorse
- 1.3 Scelte Tecnologiche
- 1.4 Implementazione
  - 1.4.1 Definizione del Docker-Compose
  - 1.4.2 Preparazione directory del cluster
  - 1.4.3 Configurazione iniziale dei file di servizio
  - 1.4.4 Verifica funzionamento
- 1.5 Integrazione HIDS Agentless (Log Collection)

Capitolo 2: Configurazione della Zona Autoritativa e Baseline di Monitoraggio
- 2.1 Definizione della Zona e Canale di Controllo
- 2.2 Il Database della Zona
  - 2.2.1 Approfondimento: Il Record di tipo A (Address)
- 2.3 Validazione del Canale di Monitoraggio Agentless
- 2.4 Ampliamento della Superficie di Attacco
  - File di zona aggiornato (`./bind/zones/db.progetto.ns` - Serial 3):

Capitolo 3: Simulazione attacco DNS Cache Poisoning
- 3.1 Setup della Baseline Forense
- 3.2 Espansione dell'Infrastruttura di Attacco
  - 3.2.1 Il Nodo Attaccante (Kali Linux)
  - 3.2.2 Il Sito Clone (Web Masquerade)
- 3.3 Validazione della Connettività e Reconnaissance
- 3.4 Weaponization
  - Logica dello script (`dns_poison.py`):
- 3.5 Esecuzione dell'attacco
  - 3.5.1 War Room
  - 3.5.2 Esito dell'attacco
- 3.6 Modifica architettura
  - 3.6.1 Aggiunta Container e configurazione Resolver
  - 3.6.2 Infrastruttura di rete aggiornata
- 3.7 Preparazione nuovo attacco
  - 3.7.1 Limiti dello Spoofing in Ambiente Containerizzato

3.8 Cambio di Prospettiva - Abuso di Dynamic DNS Update non Autenticato
3.8.1 Modello di Minaccia
3.8.2 Estensione della visibilità sui log e configurazione IDS
3.8.3 Esecuzione dell'attacco
Fase 1 – Reconnaissance
Fase 2 – Verifica Superficie di Attacco
Fase 3 – Exploitation
Capitolo 4: Detection & Analysis
4.1 Rilevamento dell'Anomalia tramite HIDS (Wazuh)
4.2 Analisi Forense del Traffico di Rete (PCAP)
4.3 Impatto sulla Cache del Resolver
4.4 Considerazioni Architetturali
Capitolo 5: Hardening
5.1 Eradication & Recovery
Fase 1 - Congelamento Zona sul Master
Fase 2 - Sostituzione del file di zona
Fase 3 - Scongelamento e reload della zona
Fase 4 - Flush della cache del Resolver
Fase 5 - Verifica della pulizia
5.2 Hardening Diretto (TSIG e ACL)
Fase 1 - Generazione del Segreto Crittografico
Fase 2 - Messa in Sicurezza del Server Master (Backend)
Fase 3 - Hardening del Resolver e Defense-in-Depth
Fase 4 - Validazione delle Contromisure (Test di Sicurezza)
5.3 Hardening Strutturale (DNSSEC)
Il ruolo delle chiavi crittografiche
5.3.1 Architettura della Firma (Master Side)
5.3.2 Configurazione della Validazione (Master e Resolver)
5.3.3 Gestione dell'eccezione locale (Resolver Side)
5.3.4 Verifica dell'Integrità Crittografica
5.4 Risultati Ottenuti
Capitolo 6: Validation & Analysis
6.1 Fase 1: Neutralizzazione dell'Attacco
Analisi Forense del Traffico tramite Wireshark
6.2 Fase 2: Amministrazione Sicura
Analisi Forense del Traffico tramite Wireshark
6.3 Fase 3: Validazione Crittografica e Risoluzione DNSSEC
6.3.1 Step 1: Proof of Signing (Network: Management)
6.3.2 Step 2: Proof of Validation (Network: Frontend)
Considerazioni sulla Sicurezza
Analisi Forense del Traffico tramite Wireshark
6.4 Fase 4: Analisi delle Segnalazioni (SIEM Monitoring)
Considerazioni Architetturali e Necessità di Tuning (Detection Engineering)
6.5 Sintesi dei Risultati di Collaudo
Capitolo 7: Conclusioni e Sviluppi Futuri
7.1 Sintesi del Progetto e Obiettivi Raggiunti
7.2 Limiti dell'Ambiente e Lezioni Apprese (Lessons Learned)
7.3 Sviluppi Futuri

Introduzione

Il **Domain Name System (DNS)** rappresenta la spina dorsale dell'attuale infrastruttura Internet, agendo come un elenco telefonico globale che traduce nomi a dominio leggibili in indirizzi IP numerici. Nonostante la sua criticità, il protocollo originale è stato progettato prediligendo l'efficienza rispetto alla sicurezza, risultando intrinsecamente privo di meccanismi di autenticazione e integrità.

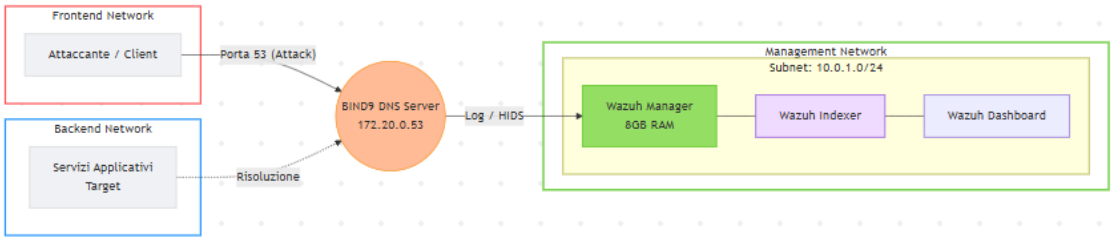
Tale lacuna espone il sistema a minacce significative, come il **Cache Poisoning**, che compromettono l'integrità dei dati (uno dei pilastri della triade CIA) e possono portare al dirottamento del traffico verso risorse malevole. Il presente progetto si propone di analizzare queste vulnerabilità attraverso una simulazione pratica di attacco e di implementare una strategia di difesa multilivello basata sulle estensioni **DNSSEC** e su sistemi di **Intrusion Detection (IDS)** come Wazuh, al fine di stabilire una catena di fiducia (Chain of Trust) e monitorare attivamente le anomalie di rete.

Capitolo 1: Progettazione e Setup dell'Infrastruttura

1.1 Architettura della Rete

Per garantire l'isolamento dei servizi e simulare un ambiente aziendale realistico, l'infrastruttura è stata suddivisa in tre segmenti di rete logici (Docker Networks):

- **Management Network (10.0.1.0/24):** Utilizzata per il traffico di controllo tra Wazuh Manager, Indexer e le dashboard. È una rete isolata dal traffico pubblico.
- **Frontend Network (172.20.0.0/24):** Destinata all'esposizione del servizio DNS (Porta 53) verso i client e, in fase successiva, verso l'attaccante.
- **Backend Network (192.168.100.0/24):** Utilizzata per la comunicazione interna tra i servizi applicativi



1.2 Dimensionamento delle Risorse

Data la natura dell'IDS **Wazuh**, che richiede una gestione intensiva di Java per l'indicizzazione dei log (Elasticsearch/OpenSearch), sono state allocate le seguenti risorse minime:

- **Wazuh Stack (Manager, Indexer, Dashboard)**: 10GB RAM totali consigliati (8GB per l'elaborazione e l'indicizzazione, 2GB per l'interfaccia di visualizzazione).
- **BIND9 Server:** 512MB di RAM (leggero ed efficiente).

1.3 Scelte Tecnologiche

- **Containerizzazione:** Si è scelto Docker per garantire la riproducibilità dell'ambiente e la facilità di rollback in caso di errori durante la firma delle zone DNSSEC.
- **Sistema Operativo Base:** Ubuntu-based images per una maggiore compatibilità con i tool di analisi.
- L'architettura scelta si basa sullo **stack moderno** di Wazuh, dove l'**Indexer** (basato su OpenSearch) sostituisce Elasticsearch per lo storage dei dati. Per mantenere il server BIND9 leggero e sicuro, si è optato per un monitoraggio **agentless**. Invece di installare un agent nel container DNS, il Manager analizza i log tramite il volume condiviso `./bind/log`, garantendo che l'HIDS possa rilevare anomalie senza interferire con le prestazioni del servizio DNS.

> Per il monitoraggio dell'infrastruttura è stato scelto **Wazuh**, una piattaforma XDR/SIEM di tipo host-based (HIDS). Sebbene la minaccia analizzata (DNS Poisoning) sia network-sided, la detection verrà affidata all'analisi granulare dei log applicativi del server BIND9 e al monitoraggio dell'integrità dei file di zona, permettendo una visibilità profonda sugli eventi interni al server che un NIDS tradizionale potrebbe ignorare se il traffico fosse cifrato o incapsulato.

1.4 Implementazione

1.4.1 Definizione del Docker-Compose

Per la realizzazione del laboratorio è stato redatto un file `docker-compose.yml` volto a orchestrare i servizi core: **BIND9** (DNS), **Wazuh-Manager** e **Wazuh-Indexer**.

Il file `docker-compose.yml` è stato progettato per riflettere la segmentazione di rete descritta. Oltre alla definizione delle tre network isolate (Frontend, Backend, Management), sono stati implementati i seguenti parametri critici:

- **Assegnazione IP Statici**: Necessaria per garantire la persistenza dei puntamenti DNS e la stabilità delle comunicazioni verso il manager IDS.

Servizio	Rete	IP Statico	Ruolo Tecnico
BIND9 Server	Frontend	`172.20.0.53`	Esposizione servizio DNS (Porta 53)
BIND9 Server	Management	`10.0.1.53`	Interfaccia per Logging/HIDS
Wazuh Manager	Management	`10.0.1.10`	Analisi comportamentale e SIEM
Wazuh Indexer	Management	`10.0.1.11`	Database indicizzazione alert
Wazuh Dashboard	Management	`10.0.1.12`	Interfaccia grafica (Visualizzazione Alert)

- **Gestione Volumi**: Sono stati creati dei *bind mount* per permettere a Wazuh di accedere ai log di BIND9 (`/var/log/named/query.log`) in modalità sola lettura, abilitando la funzione di log-analysis dell'HIDS garantendo continuità del servizio
- **Allocazione Risorse**: È stata rispettata la specifica di allocazione di 8GB di RAM per il container Wazuh-Manager per supportare l'elaborazione dei flussi di log.
- **Configurazione Dual-Homed**: Il container **BIND9** è stato configurato con una doppia interfaccia di rete (dual-homed), collegandolo simultaneamente alla *Frontend Network* (per il servizio pubblico) e

alla *Management Network* (per l'invio sicuro dei log all'IDS). Questa segregazione garantisce che il traffico di monitoraggio sia isolato da quello delle query DNS, riducendo la superficie di attacco del piano di gestione.

- **Configurazione porte:** è stata limitata allo stretto necessario per ridurre la superficie di attacco. In particolare, sono state esposte le porte **53/UDP-TCP** per la fornitura del servizio DNS e la porta **443** per l'accesso alla dashboard amministrativa, mantenendo tutti gli altri flussi di comunicazione (come quelli verso l'indexer) segregati all'interno della rete di management.
- L'accesso all'interfaccia di monitoraggio (**Wazuh Dashboard**) è riservato esclusivamente al personale amministrativo. Per simulare questo isolamento, la Dashboard è attestata sulla **Management Network**, rendendola invisibile ai nodi presenti nella *Frontend Network* (dove opererà l'attaccante). L'esposizione del servizio avviene tramite un tunnel protetto sulla porta 443 dell'host, garantendo la segregazione tra il traffico di servizio DNS e le operazioni di sicurezza.

1.4.2 Preparazione directory del cluster

Prima di poter utilizzare Docker Compose, è necessario avere la seguente struttura di cartelle per permettere ai vari container di funzionare correttamente:

```
./progetto-ns/
├── docker-compose.yml
├── bind/
│   ├── config/
│   ├── zones/
│   └── log/
└── wazuh/
    └── config/
```

1.4.3 Configurazione iniziale dei file di servizio

Per assicurare un avvio coerente del servizio BIND9, all'interno della directory `/bind/config` sono stati predisposti i file di configurazione minimi necessari:

1. `named.conf`: configurato per includere i file di opzioni e le zone locali.
2. `named.conf.options`: dove vanno definite le ACL, le porte di ascolto e i parametri di sicurezza globali. La configurazione è stata raffinata per garantire che il server operi come Autoritativo, riflettendo uno scenario reale in cui i dipendenti di un'organizzazione utilizzano il DNS aziendale per navigare verso domini esterni:

```
options {
        directory "/var/cache/bind";

        // Ascolto su tutte le interfacce (Frontend e Management)
        listen-on { any; };

        // Per permettere l'esecuzione di query ricorsive
```

```
        recursion yes;

        // Controllo accessi: permette query da qualsiasi client
        allow-query { any; };

        // Validazione DNSSEC disabilitata temporanteamente
        dnssec-validation no;
};
```

3. In particolare, è stato implementato un canale di logging dedicato (`query_log`). Questa configurazione istruisce il demone BIND9 a registrare ogni transazione DNS nel file `/var/log/named/query.log`. Questa aggiunta trasforma il server DNS in una sorgente di dati leggibile dall'HIDS.
   a. È stata inoltre generata la chiave `rndc.key` (tramite il comando `docker exec bind9_server rndc-confgen -a`) all'interno del volume di configurazione per permettere la gestione sicura del servizio senza accessi diretti al container
4. `named.conf.local`: predisposto per la dichiarazione delle zone autoritative

> Questi file sono stati configurati in modo minimale in modo tale da far partire correttamente il container di BIND9, successivamente verranno aggiornati

1.4.4 Verifica funzionamento

Una volta terminata la fase preparatoria, è possibile eseguire il comando `docker-compose up -d` per attivare il cluster Docker.

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker compose up -d
[+] Running 12/12
 ✔wazuh_dashboard Pulled                                                                                                    49.2s
   ✔4320f72af303 Pull complete                                                                                               0.6s
   ✔c8fb93ba3005 Pull complete                                                                                              47.5s
   ✔6080756e33a6 Pull complete                                                                                               0.6s
   ✔4ad3eea92d60 Pull complete                                                                                              47.4s
   ✔157702938a2e Pull complete                                                                                               0.7s
   ✔5c4c52aa0571 Pull complete                                                                                               0.7s
   ✔4f4fb700ef54 Pull complete                                                                                               0.7s
   ✔7fecafd1416d Pull complete                                                                                               0.7s
   ✔958b6a9da1cf Pull complete                                                                                               0.9s
   ✔17c22431cbe4 Pull complete                                                                                               0.8s
   ✔1631136f8eb3 Pull complete                                                                                               0.7s
[+] Running 6/6
 ✔Network progetto-ns_mgmt_net      Created                                                                                  0.1s
 ✔Network progetto-ns_frontend_net  Created                                                                                  0.1s
 ✔Container wazuh_manager           Started                                                                                  1.3s
 ✔Container wazuh_indexer           Started                                                                                  1.3s
 ✔Container bind9_server            Started                                                                                  1.8s
 ✔Container wazuh_dashboard         Started                                                                                  1.9s
```

Ora bisogna effettuare vari controlli per controllare se l'infrastruttura è funzionante:

1. **Check Processi**: `docker ps` per verificare che tutti i container siano UP

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED          STATUS          PORTS                                          NAMES
47772679ac12   wazuh/wazuh-dashboard:4.7.2    "/entrypoint.sh"         55 seconds ago   Up 53 seconds   0.0.0.0:443->5601/tcp, [::]:443->5601/tcp      wazuh_das
hboard
e8c447d814d5   ubuntu/bind9                   "docker-entrypoint.sh"   55 seconds ago   Up 54 seconds   53/tcp, 953/tcp, 53/udp                        bind9_ser
ver
25c2a152759e   wazuh/wazuh-indexer:4.7.2      "/entrypoint.sh open…"   55 seconds ago   Up 54 seconds   9200/tcp                                       wazuh_ind
exer
bcafd1b8c9ac   wazuh/wazuh-manager:4.7.2      "/init"                  55 seconds ago   Up 54 seconds   1514-1516/tcp, 514/udp, 55000/tcp              wazuh_man
ager
```

2. **Verifica Dual-Homing BIND9**: `docker exec bind9_server ip addr`

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec bind9_server ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0@if115: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether f2:28:17:28:37:25 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.20.0.53/24 brd 172.20.0.255 scope global eth0
       valid_lft forever preferred_lft forever
3: eth1@if116: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 8e:d0:0a:85:e4:cb brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.0.1.53/24 brd 10.0.1.255 scope global eth1
       valid_lft forever preferred_lft forever
```

3. **Verifica adiacenza BIND9 e IDS**: `docker network inspect progetto-ns_mgmt_net`

```
        "Containers": {
            "7c6ad98c8d4b99b4b57726fadfc95deba48a01774d8b2b6ff862ff2d97f3e9fa": {
                "Name": "wazuh_indexer",
                "EndpointID": "8ffeb4f6e17760c0ec4d6c698151b5b535f77b84606474da7d5bdcd970d9cb08",
                "MacAddress": "32:6a:8a:34:87:09",
                "IPv4Address": "10.0.1.11/24",
                "IPv6Address": ""
            },
            "988ba795f27d34c713a7583f6892908413bf00c1445c194eedcd441ec94bb93e": {
                "Name": "wazuh_manager",
                "EndpointID": "4b3348ccefc8dd52882eca7932a22c38b65add05223ef2356b37806f6a0ade4b",
                "MacAddress": "0e:9c:51:14:59:3f",
                "IPv4Address": "10.0.1.10/24",
                "IPv6Address": ""
            },
            "ee858eda2410fa7672eab7f93bd8b42b965626d5dd3b8d588e1a6209244cc7c7": {
                "Name": "bind9_server",
                "EndpointID": "88f7da45a4ce5e6f085656f5fc7ff46cbe21fbf10f7be817ed3f4d1c7517ba4e",
                "MacAddress": "8e:d0:0a:85:e4:cb",
                "IPv4Address": "10.0.1.53/24",
                "IPv6Address": ""
            }
        },
```

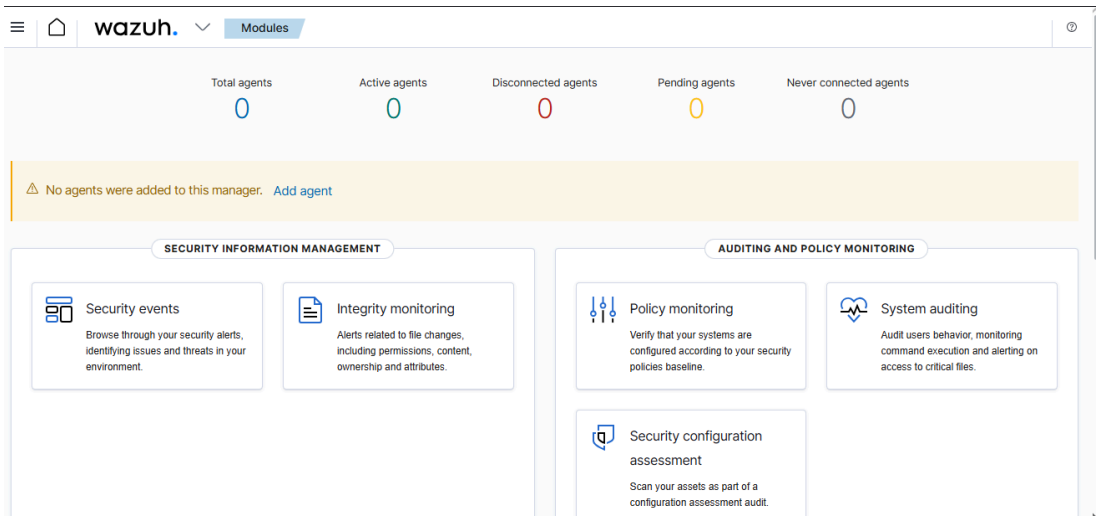
Data l'assenza di utility non necessarie all'interno delle immagini minimali (scelta di *hardening* per ridurre la superficie di attacco), la connettività è stata confermata verificando l'adiacenza dei nodi sul bridge Docker. La comunicazione tra il server DNS e il Manager IDS è stata validata con successo, garantendo che il canale per l'invio dei log (fondamentale per le fasi successive del progetto) sia operativo.

4. **Verifica Generazione Log e Persistenza:**
   a. `docker exec bind9_server rndc reload`, questo comando dice a BIND9 di ricaricare i file, quindi è per generare per la prima volta il file `query.log` senza dover generare traffico fittizio iniziale
   b. `docker exec bind9_server ls -l /var/log/named/`

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec bind9_server ls -l /var/log/named/
total 0
-rw-r--r-- 1 bind bind 0 Feb 20 22:48 query.log
```

Grazie a questi comandi è stata confermata la creazione del file `query.log` nel volume condiviso `./bind/log/`. Il test ha avuto successo, validando la capacità del server di produrre metadati in tempo reale e confermando che il canale di comunicazione verso Wazuh è operativo e persistente.

5. **Verifica funzionamento Dashboard:** `http://localhost:443`



Parentesi sulla risoluzione dei Conflitti SSL

Durante il primo bootstrap, si è riscontrato un loop di errori critici (`bad_certificate` e `NotSslRecordException`). L'analisi dei log ha evidenziato che, nonostante la configurazione "demo", i componenti Manager e Dashboard tentavano di negoziare connessioni TLS su porte non correttamente inizializzate.

- **Disattivazione del Security Plugin**: È stato forzato il parametro `plugins.security.disabled: true` all'interno del file `opensearch.yml` dell'Indexer.
- **Migrazione a protocollo HTTP**: La comunicazione tra Manager, Indexer e Dashboard è stata spostata su HTTP, eliminando l'overhead dei certificati.

> ⚠️ Disattivare i plugin di sicurezza non è la mossa migliore, tuttavia tale scelta è stata validata dal fatto che tutto il traffico di management sensibile è confinato nella rete **10.0.1.0/24**, isolata logicamente dal piano di attacco.

1.5 Integrazione HIDS Agentless (Log Collection)

Per rendere operativa la scelta tecnologica descritta nel punto 1.3, è necessario istruire il Wazuh Manager a monitorare il file prodotto da BIND9 all'interno del volume condiviso.

Operazione sul Manager:
Bisogna modificare la configurazione del Manager affinché legga il file che BIND9 deposita in `/var/log/named/query.log`. Per il Manager, tale risorsa è disponibile nel percorso di mount `/var/log/remote_bind/query.log` definito nel descrittore di deployment.

```
#Modifica configurazione
docker exec -it wazuh_manager bash -c "echo -e '\n<ossec_config>\n  <localfil
e>\n    <log_format>syslog</log_format>\n    <location>/var/log/remote_bind/q
uery.log</location>\n  </localfile>\n</ossec_config>' >> /var/ossec/etc/osse
c.conf"
```

```
#Riavvio interno per applicare modifiche
docker exec -it wazuh_manager /var/ossec/bin/wazuh-control restart
```

Validazione del mount point: `docker exec -it wazuh_manager ls -l /var/log/remote_bind/query.log`

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it wazuh_manager ls -l /var/log/remote_bind/query.log
-rw-r--r-- 1 _apt wazuh 0 Feb 20 22:48 /var/log/remote_bind/query.log
```

L'operazione ha avuto successo, confermando che il flusso di dati tra il piano di servizio (DNS) e il piano di gestione (IDS) è attivo in modalità agentless. La fase di configurazione e setup dell'infrastruttura può definirsi conclusa.

Capitolo 2: Configurazione della Zona Autoritativa e Baseline di Monitoraggio

In questa fase viene definita la zona autoritativa `progetto.ns`, configurata per operare sulla rete di Frontend (**172.20.0.53**). L'obiettivo è stabilire una base operativa solida per il monitoraggio dei log applicativi, garantendo che ogni risoluzione di nomi generi un evento tracciabile.

2.1 Definizione della Zona e Canale di Controllo

Per la gestione del demone `named`, è stato configurato il canale di controllo **RNDC**. Questo permette di amministrare il server DNS in modo sicuro tramite crittografia a chiave condivisa, facilitando operazioni di manutenzione senza interruzione del servizio.

Configurazione di `named.conf.local`:

```
// Definizione della zona autoritativa
zone "progetto.ns" {
type master;
file "/var/lib/bind/db.progetto.ns";
allow-query { any; };
};
```

2.2 Il Database della Zona

Il file di zona è stato strutturato per mappare i servizi critici dell'infrastruttura. È stato impostato un valore di **Serial** incrementale per la gestione delle versioni e definiti i record di tipo **A** per il name server e il servizio web simulato.

File di zona `./bind/zones/db.progetto.ns`:

```
$TTL    604800
@       IN      SOA     ns1.progetto.ns. admin.progetto.ns. (
                              2         ; Serial (Versione della zona)
                         604800         ; Refresh
```

```
                            86400         ; Retry
                          2419200         ; Expire
                           604800 )       ; Negative Cache TTL

@       IN      NS      ns1.progetto.ns.
ns1     IN      A       172.20.0.53
www     IN      A       1.2.3.4
```

2.2.1 Approfondimento: Il Record di tipo A (Address)

Il record di tipo **A** (Address) è la risorsa più comune all'interno del Domain Name System. La sua funzione principale è quella di mappare un nome a dominio (hostname) verso un indirizzo **IPv4** a 32 bit.

All'interno di questo progetto, il record A svolge un ruolo cruciale per diverse ragioni:

- **Risoluzione dei Nomi**: Permette la traduzione di nomi simbolici leggibili dall'utente, come `www.progetto.ns`, nell'indirizzo IP numerico `1.2.3.4` necessario per l'instradamento dei pacchetti sulla rete.
- **Definizione dei Servizi**: Viene utilizzato per identificare i punti di accesso ai servizi critici, inclusi il Name Server stesso (`ns1`), i server di posta (`mail`) e i gateway di accesso remoto (`vpn`).
- **Puntamento all'Infrastruttura**: Collega i nomi definiti nel file di zona agli IP statici configurati all'interno della rete Docker, garantendo che le query dei client vengano dirette correttamente verso i container di destinazione

> **Nota**: In un attacco di **Cache Poisoning** (oggetto del Capitolo 3), l'obiettivo dell'attaccante è proprio quello di falsificare un record di tipo A, sovrascrivendo l'IP legittimo con uno malevolo per dirottare il traffico.

2.3 Validazione del Canale di Monitoraggio Agentless

Il "ponte" tra il server DNS e il sistema HIDS (Wazuh) si basa sulla visibilità dei log nel file `query.log`. In questa architettura, Wazuh opera in modalità **agentless**, analizzando direttamente i flussi di testo generati da BIND9.

- **Restart BIND9 per caricare la zona:** `docker compose restart bind9_server`

```
21-Feb-2026 16:24:33.820 command channel listening on ::1#953
21-Feb-2026 16:24:33.820 managed-keys-zone: loaded serial 11
21-Feb-2026 16:24:33.830 /var/lib/bind/db.progetto.ns:11: file does not end with newline
21-Feb-2026 16:24:33.830 zone progetto.ns/IN: loaded serial 2
21-Feb-2026 16:24:33.830 all zones loaded
21-Feb-2026 16:24:33.830 FIPS mode is disabled
```

Dopo ciò stato necessario superare due criticità emerse durante la fase di setup:

1. **Mappatura delle Categorie**: Nonostante la definizione del canale di output, è stato fondamentale istruire BIND9 a inviare le query specifiche a tale canale tramite l'istruzione `category queries`. Senza questa associazione, il demone non inizializza il descrittore di file per le transazioni DNS.

2. **Risoluzione del conflitto di Logging (Flag -g)**: Si è riscontrato che l'immagine Docker predefinita avviava il processo con il flag `g`, che forza l'invio dei log allo *standard error* ignorando le direttive del file di configurazione. La soluzione ha previsto la modifica dell'**entrypoint** nel file `docker-compose.yml` per utilizzare il flag `f`, garantendo la persistenza dei log su file fisico.

Configurazione finale del blocco logging:

```
logging {
    channel query_log {
        file "/var/log/named/query.log" versions 3 size 5m;
        severity info;
        print-time yes;
        print-category yes;
        print-severity yes;
    };
    category queries { query_log; };
};
```

Una volta corretti i parametri di avvio e la sintassi del file di zona (assicurando la presenza di una riga vuota finale per il corretto parsing del record WWW), è stato possibile testare effettivamente il server DNS.

- **Test di risoluzione:** `nslookup www.progetto.ns 127.0.0.1`

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>nslookup www.progetto.ns 127.0.0.1
Server:  UnKnown
Address:  127.0.0.1

Nome:    www.progetto.ns
Address:  1.2.3.4
```

> **Nota**: per questo test, condotto da macchina Windows, è stato utilizzato l'indirizzo di loopback `127.0.0.1`. Ciò è dovuto alla natura di Docker Desktop, che espone il servizio DNS tramite mappatura delle porte (`53:53`), rendendolo accessibile all'host esterno senza dover configurare rotte statiche verso il bridge Docker. Successivamente verrà usato l'IP della rete Frontend

- **Test generazione log:** `docker exec -u 0 -it bind9_server cat /var/log/named/query.log`

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -u 0 -it bind9_server cat /var/log/named/query.log

21-Feb-2026 21:21:37.689 queries: info: client @0x7fc30601d000 172.20.0.1#47021 (1.0.0.127.in-addr.arpa): query: 1.0.0.127.in-addr.arpa IN PTR + (172.20.0.5
3)
21-Feb-2026 21:21:37.699 queries: info: client @0x7fc30601d000 172.20.0.1#37024 (www.progetto.ns): query: www.progetto.ns IN A + (172.20.0.53)
21-Feb-2026 21:21:37.699 queries: info: client @0x7fc304b42000 172.20.0.1#60030 (www.progetto.ns): query: www.progetto.ns IN AAAA + (172.20.0.53)
21-Feb-2026 21:23:08.495 queries: info: client @0x7fc304b42000 172.20.0.1#50956 (1.0.0.127.in-addr.arpa): query: 1.0.0.127.in-addr.arpa IN PTR + (172.20.0.5
3)
```

- **Test Canale di monitoraggio:** `docker exec -it wazuh_manager cat /var/log/remote_bind/query.log`

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it wazuh_manager cat /var/log/remote_bind/query.log
22-Feb-2026 11:11:30.037 queries: info: client @0x7f0673c2c000 10.0.1.60#58313 (www.progetto.ns): query: www.progetto.ns
 IN A +E(0)DCK (10.0.1.53)
22-Feb-2026 11:11:30.048 queries: info: client @0x7f0678e2b000 10.0.1.60#34498 (www.progetto.ns): query: www.progetto.ns
 IN AAAA +E(0)DCK (10.0.1.53)
```

2.4 Ampliamento della Superficie di Attacco

Sebbene un singolo record `A` sia sufficiente per dimostrare un attacco di *Cache Poisoning*, per rendere il progetto realistico e pienamente conforme alle fasi successive (in particolare quella relativa all'hardening), è stato deciso di popolare la zona `progetto.ns` con ulteriori record

File di zona aggiornato (`./bind/zones/db.progetto.ns` - Serial 3):

```
$TTL    604800
@       IN      SOA     ns1.progetto.ns. admin.progetto.ns. (
                              3         ; Serial (Versione aggiornata)
                         604800          ; Refresh
                          86400          ; Retry
                        2419200          ; Expire
                         604800 )        ; Negative Cache TTL

@       IN      NS      ns1.progetto.ns.
ns1     IN      A       172.20.0.53
www     IN      A       1.2.3.4
mail    IN      A       1.2.3.5         ; Server di posta aziendale
vpn     IN      A       1.2.3.6         ; Gateway per accesso remoto
db      IN      A       192.168.100.10  ; Database interno (Backend Network)
```

L'inserimento di questi record risponde a precise necessità metodologiche del progetto:

- **Simulazione di Ricognizione**: Fornisce un set di dati realistico per le attività di *enumeration* tramite tool come `dig` e `host`, permettendo all'attaccante di mappare una superficie di attacco non banale.
- **Validazione DNSSEC**: La presenza di più host è fondamentale per testare l'efficacia dei record **NSEC3** (che verrà implementato successivamente) contro le tecniche di *Zone Walking*. Con un solo record, la dimostrazione della protezione crittografica contro l'enumerazione forzata risulterebbe priva di valore analitico.
- **Segregazione Logica**: L'inserimento di un record che punta alla `Backend Network` (192.168.100.10) permette di verificare come il DNS possa esporre informazioni su segmenti di rete teoricamente isolati.

> **Nota sull'aggiornamento del Serial**
>
> Il numero seriale del record SOA (Start of Authority) è stato incrementato da **2** a **3** per segnalare formalmente al demone BIND9 che il database di zona ha subito delle modifiche.
>
> Nel protocollo DNS, il campo *Serial* funge da numero di versione: un valore superiore a quello precedentemente caricato è il meccanismo standard che istruisce il server a ricaricare i dati aggiornati. Questa procedura è fondamentale per garantire che i nuovi record siano immediatamente attivi per la risoluzione e che le relative query vengano correttamente tracciate dal sistema di logging per la baseline di monitoraggio.

> ❗ Le ulteriori verifiche per il corretto funzionamento dell'aggiunta sono state condotte off-documentazione, al fine di non appesantire la trattazione con informazioni ridondanti, riportando nel presente report esclusivamente la configurazione finale validata e funzionante.

Con quest'ultimo passaggio è terminata la configurazione della Zona Autoritativa ed è stato validato il corretto funzionamento della baseline di monitoraggio, è possibile ora passare allo step successivo. L'infrastruttura dispone ora di una sorgente dati affidabile e di un sistema HIDS (Wazuh) correttamente integrato in modalità agentless. Ogni query generata è ora tracciata, stabilendo il punto di partenza ideale per la rilevazione delle anomalie durante le fasi di attacco.

Capitolo 3: Simulazione attacco DNS Cache Poisoning

L'obiettivo di questa fase è testare la resilienza dell'infrastruttura e la capacità di detection del sistema SIEM attraverso la simulazione di un attacco di **DNS Cache Poisoning**. L'attaccante mira a inquinare la cache del resolver per dirottare le richieste legittime verso un'infrastruttura malevola controllata (**Masquerade**).

3.1 Setup della Baseline Forense

Prima di procedere con l'attività offensiva, è stato fondamentale documentare lo stato "sano" della rete. Questa procedura è necessaria per avere un termine di paragone durante l'analisi forense e per convalidare il comportamento standard dei protocolli.

Per l'acquisizione del traffico sono stati effettuati i seguenti passaggi:

- è stato installato il tool `tcpdump` all'interno del container `bind9_server`:

  ```
  docker exec -u 0 -it bind9_server bash -c "apt-get update && apt-get insta
  ll -y tcpdump"
  ```

- È stata avviata una cattura sulla porta 53, registrando le query legittime effettuate dall'host Windows verso i record definiti nel Capitolo 2 (es. `www.progetto.ns`, `mail.progetto.ns`):

  ```
  docker exec bind9_server tcpdump -i any port 53 -w /var/log/named/legit_tr
  affic.pcap
  ```

  - Le query effettuate sono state di vario tipo, per generare traffico legittimo di ogni tipo, da query per record di tipo A, a record strutturali, fino arrivare a "errori legittimi"

  ```
  nslookup www.progetto.ns 127.0.0.1
  nslookup mail.progetto.ns 127.0.0.1
  nslookup vpn.progetto.ns 127.0.0.1
  nslookup db.progetto.ns 127.0.0.1
  nslookup -type=NS progetto.ns 127.0.0.1
  ```

```
nslookup -type=SOA progetto.ns 127.0.0.1
nslookup errore.progetto.ns 127.0.0.1
```

Il file risultante, `legit_traffic.pcap`, mostra una risoluzione coerente con i Transaction ID (TXID) generati correttamente e risposte autoritative puntuali.

24	19.746413	172.20.0.53	172.20.0.1	DNS	126	Standard query response 0x0003 AAAA db.progetto.ns SOA ns1.progetto.ns
25	24.454065	172.20.0.1	172.20.0.53	DNS	88	Standard query 0x0001 PTR 1.0.0.127.in-addr.arpa
26	24.455347	172.20.0.53	172.20.0.1	DNS	88	Standard query response 0x0001 Refused PTR 1.0.0.127.in-addr.arpa
27	24.458918	172.20.0.1	172.20.0.53	DNS	77	Standard query 0x0002 NS progetto.ns
28	24.460319	172.20.0.53	172.20.0.1	DNS	111	Standard query response 0x0002 NS progetto.ns NS ns1.progetto.ns A 172.20.0.53
29	29.839905	172.20.0.1	172.20.0.53	DNS	88	Standard query 0x0001 PTR 1.0.0.127.in-addr.arpa
30	29.841229	172.20.0.53	172.20.0.1	DNS	88	Standard query response 0x0001 Refused PTR 1.0.0.127.in-addr.arpa
31	29.844990	172.20.0.1	172.20.0.53	DNS	77	Standard query 0x0002 SOA progetto.ns
32	29.846144	172.20.0.53	172.20.0.1	DNS	123	Standard query response 0x0002 SOA progetto.ns SOA ns1.progetto.ns
33	36.068281	172.20.0.1	172.20.0.53	DNS	88	Standard query 0x0001 PTR 1.0.0.127.in-addr.arpa
34	36.069911	172.20.0.53	172.20.0.1	DNS	88	Standard query response 0x0001 Refused PTR 1.0.0.127.in-addr.arpa
35	36.074212	172.20.0.1	172.20.0.53	DNS	84	Standard query 0x0002 A errore.progetto.ns
36	36.075424	172.20.0.53	172.20.0.1	DNS	130	Standard query response 0x0002 No such name A errore.progetto.ns SOA ns1.progetto.ns
37	36.077803	172.20.0.1	172.20.0.53	DNS	84	Standard query 0x0003 AAAA errore.progetto.ns
38	36.078765	172.20.0.53	172.20.0.1	DNS	130	Standard query response 0x0003 No such name AAAA errore.progetto.ns SOA ns1.progetto.ns
39	36.080905	172.20.0.1	172.20.0.53	DNS	84	Standard query 0x0004 A errore.progetto.ns
40	36.081925	172.20.0.53	172.20.0.1	DNS	130	Standard query response 0x0004 No such name A errore.progetto.ns SOA ns1.progetto.ns
41	36.083862	172.20.0.1	172.20.0.53	DNS	84	Standard query 0x0005 AAAA errore.progetto.ns

> **Nota**: nel file pcap si nota che la comunicazione avviene sull'interfaccia della rete Frontend dell'infrastruttura nonostante abbiamo usato l'indirizzo Loopback della macchina a causa del Bridge di Docker Desktop, come già specificato in precedenza

3.2 Espansione dell'Infrastruttura di Attacco

Per riflettere uno scenario di minaccia reale, il file `docker-compose.yml` è stato aggiornato introducendo due nuovi nodi all'interno della **Frontend Network** (`172.20.0.0/24`).

3.2.1 Il Nodo Attaccante (Kali Linux)

Viene utilizzato un container basato su **Kali Rolling** (`attacker`). Per permettere la manipolazione dei pacchetti a basso livello necessaria agli script di attacco, è stata assegnata la capability `NET_ADMIN`. Il nodo è configurato con IP statico `172.20.0.100`.

3.2.2 Il Sito Clone (Web Masquerade)

Per simulare il successo dell'attacco, è stato distribuito un server **Nginx** (`web_clone`) con IP `172.20.0.80`. Tramite un bind mount, il server espone una pagina HTML contraffatta che conferma visivamente l'avvenuto dirottamento del traffico.

3.3 Validazione della Connettività e Reconnaissance

Una volta attivati i nuovi servizi (`docker compose up -d attacker web_clone`), è stata verificata la raggiungibilità del target dal punto di vista dell'attaccante:

1. **Test di Raggiungibilità**: `docker exec -it kali_attacker ping -c 4 172.20.0.53`

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it kali_attacker ping -c 4 172.20.0.53
PING 172.20.0.53 (172.20.0.53) 56(84) bytes of data.
64 bytes from 172.20.0.53: icmp_seq=1 ttl=64 time=0.582 ms
64 bytes from 172.20.0.53: icmp_seq=2 ttl=64 time=0.047 ms
64 bytes from 172.20.0.53: icmp_seq=3 ttl=64 time=0.056 ms
64 bytes from 172.20.0.53: icmp_seq=4 ttl=64 time=0.043 ms

--- 172.20.0.53 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3102ms
rtt min/avg/max/mdev = 0.043/0.182/0.582/0.230 ms
```

- Il comando ha confermato la corretta adiacenza tra il nodo Kali e il server DNS sulla rete Frontend.

2. **Verifica DNS "Pulita"**: `docker exec -it kali_attacker dig @172.20.0.53 www.progetto.ns`

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it kali_attacker dig @172.20.0.53 www.progetto.ns

; <<>> DiG 9.20.18-1-Debian <<>> @172.20.0.53 www.progetto.ns
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 61861
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 1373ea192cc4b2a801000000699a8755befd27ef599e1d35 (good)
;; QUESTION SECTION:
;www.progetto.ns.               IN      A

;; ANSWER SECTION:
www.progetto.ns.        604800  IN      A       1.2.3.4

;; Query time: 0 msec
;; SERVER: 172.20.0.53#53(172.20.0.53) (UDP)
;; WHEN: Sun Feb 22 04:34:29 UTC 2026
;; MSG SIZE  rcvd: 88
```

- Utilizzando il tool `dig`, è stata effettuata una query per `www.progetto.ns`. Il risultato ha confermato l'IP legittimo `1.2.3.4`, validando che la cache del server non è ancora stata compromessa.

3.4 Weaponization

L'attacco viene veicolato tramite uno script Python basato sulla libreria **Scapy**. Lo script è progettato per forgiare risposte DNS falsificate (**Spoofing**) che simulano una risposta autoritaria proveniente dal server DNS stesso.

Logica dello script (`dns_poison.py`):

- **Targeting**: Punta all'IP del server BIND9 (`172.20.0.53`).
- **Blind Spoofing**: Invia una raffica di pacchetti con Transaction ID (TXID) variabili in un range definito (1-5000). In un attacco reale, questo range sarebbe molto più ampio, ma per scopi didattici è limitato per garantire la riuscita della collisione in tempi brevi.
- **Payload Malevolo**: La risposta forgiata contiene un record di tipo **A** che associa `www.progetto.ns` all'IP del sito clone `172.20.0.80`.

```
# Snippet codice della costruzione del pacchetto malevolo
DNS(id=tx_id, qr=1, aa=1,
    qd=DNSQR(qname="www.progetto.ns."),
    an=DNSRR(rrname="www.progetto.ns.", type='A', rdata="172.20.0.80", ttl=36
00))
```

3.5 Esecuzione dell’attacco

Prima di lanciare l'attacca, è bene pulire la memoria del server DNS e il file `query.log` che sono stati sporcati con i vari test di validazione eseguiti nei capitoli precedenti:

```
docker exec bind9_server rndc flush
```

3.5.1 War Room

Per avere una visuale a 360° dell'attacco si avrà bisogno di 4 terminali:

1. **Terminale A**: Avvia la cattura pacchetti.

   ```
   docker exec bind9_server tcpdump -i any port 53 -w /var/log/named/attack_p
   hase.pcap
   ```

2. **Terminale B**: Osserva i log in tempo reale.

   ```
   docker exec bind9_server tail -f /var/log/named/query.log
   ```

3. **Terminale C**: Lancia il bombardamento Scapy.

   ```
   docker exec kali_attacker python3 /home/kali/scripts/dns_poison.py
   ```

4. **Terminale D (terminale vittima)**: Mentre lo script gira, lancia il trigger.

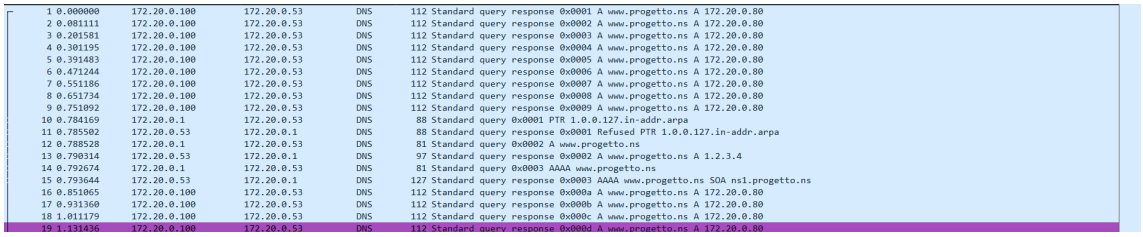
   ```
   nslookup www.progetto.ns 127.0.0.1
   ```

3.5.2 Esito dell'attacco

Analisi del traffico

Analizzando il traffico del file `attack_phase.pcap` si osserva che l'attaccante ha inviato una raffica di risposte malevole (Frame 1-9) **prima** che la vittima (Windows) generasse la query legittima (Frame 10).

- **Conseguenza**: Il server ha ricevuto risposte a domande che non aveva ancora posto.



1	0.000000	172.20.0.100	172.20.0.53	DNS	112	Standard query response 0x0001 A www.progetto.ns A 172.20.0.80
2	0.081111	172.20.0.100	172.20.0.53	DNS	112	Standard query response 0x0002 A www.progetto.ns A 172.20.0.80
3	0.201581	172.20.0.100	172.20.0.53	DNS	112	Standard query response 0x0003 A www.progetto.ns A 172.20.0.80
4	0.301195	172.20.0.100	172.20.0.53	DNS	112	Standard query response 0x0004 A www.progetto.ns A 172.20.0.80
5	0.391483	172.20.0.100	172.20.0.53	DNS	112	Standard query response 0x0005 A www.progetto.ns A 172.20.0.80
6	0.471244	172.20.0.100	172.20.0.53	DNS	112	Standard query response 0x0006 A www.progetto.ns A 172.20.0.80
7	0.551186	172.20.0.100	172.20.0.53	DNS	112	Standard query response 0x0007 A www.progetto.ns A 172.20.0.80
8	0.651734	172.20.0.100	172.20.0.53	DNS	112	Standard query response 0x0008 A www.progetto.ns A 172.20.0.80
9	0.751092	172.20.0.100	172.20.0.53	DNS	112	Standard query response 0x0009 A www.progetto.ns A 172.20.0.80
10	0.784169	172.20.0.1	172.20.0.53	DNS	88	Standard query 0x0001 PTR 1.0.0.127.in-addr.arpa
11	0.785502	172.20.0.53	172.20.0.1	DNS	88	Standard query response 0x0001 Refused PTR 1.0.0.127.in-addr.arpa
12	0.788528	172.20.0.1	172.20.0.53	DNS	81	Standard query 0x0002 A www.progetto.ns
13	0.790314	172.20.0.53	172.20.0.1	DNS	97	Standard query response 0x0002 A www.progetto.ns A 1.2.3.4
14	0.792674	172.20.0.1	172.20.0.53	DNS	81	Standard query 0x0003 AAAA www.progetto.ns
15	0.793644	172.20.0.53	172.20.0.1	DNS	127	Standard query response 0x0003 AAAA www.progetto.ns SOA ns1.progetto.ns
16	0.851065	172.20.0.100	172.20.0.53	DNS	112	Standard query response 0x000a A www.progetto.ns A 172.20.0.80
17	0.931360	172.20.0.100	172.20.0.53	DNS	112	Standard query response 0x000b A www.progetto.ns A 172.20.0.80
18	1.011179	172.20.0.100	172.20.0.53	DNS	112	Standard query response 0x000c A www.progetto.ns A 172.20.0.80
19	1.131436	172.20.0.100	172.20.0.53	DNS	112	Standard query response 0x000d A www.progetto.ns A 172.20.0.80

Ispezione pacchetti

Esaminando nel dettaglio uno dei pacchetti malevoli e uno di quelli legittimi si notano due problemi:

```
▸ Frame 1: Packet, 112 bytes on wire (896 bits), 112 bytes captured (896 bits)
▸ Linux cooked capture v2
▸ Internet Protocol Version 4, Src: 172.20.0.100, Dst: 172.20.0.53
▸ User Datagram Protocol, Src Port: 53, Dst Port: 53
▾ Domain Name System (response)
    Transaction ID: 0x0001
  ▸ Flags: 0x8500 Standard query response, No error
    Questions: 1
    Answer RRs: 1
    Authority RRs: 0
    Additional RRs: 0
  ▾ Queries
    ▸ www.progetto.ns: type A, class IN
  ▸ Answers
    [Unsolicited: True]
```

```
▸ Frame 10: Packet, 88 bytes on wire (704 bits), 88 bytes captured (704 bits)
▸ Linux cooked capture v2
▸ Internet Protocol Version 4, Src: 172.20.0.1, Dst: 172.20.0.53
▸ User Datagram Protocol, Src Port: 52675, Dst Port: 53
▾ Domain Name System (query)
    Transaction ID: 0x0001
  ▸ Flags: 0x0100 Standard query
    Questions: 1
    Answer RRs: 0
    Authority RRs: 0
    Additional RRs: 0
  ▾ Queries
    ▸ 1.0.0.127.in-addr.arpa: type PTR, class IN
    [Response In: 11]
```

- **Porta di Destinazione**: L'attaccante ha inviato i pacchetti alla porta **53** del server. Tuttavia, la query legittima della vittima si aspettava una risposta sulla porta sorgente dinamica **35239**.
- **Stato Unsolicited**: Wireshark marca correttamente il pacchetto come `[Unsolicited: True]`. Essendo il server Master per quella zona, esso non genera query ricorsive verso l'esterno per risolvere `progetto.ns`, rendendo impossibile l'accettazione di risposte non richieste.

Analisi dashboard Wazuh

Infine, la dashboard di Wazuh non ha registrato alcun alert di sicurezza ("No results for selected time range").

- **Motivazione**: Poiché i pacchetti malevoli sono stati scartati a livello di rete (mismatch di porta e stato), il demone BIND9 non ha processato le risposte e non ha generato log di errore nel file `query.log`.

Conclusioni

Il primo test ha confermato che la configurazione del server come **Authoritative Master** fornisce una protezione intrinseca contro il Cache Poisoning per i domini locali. Il server non non genera sessioni di interrogazione ricorsiva per il dominio locale per le zone di cui è autorità, non attingendo alla propria cache e scartando immediatamente ogni pacchetto che non corrisponda a una sessione attiva. Questo spiega l'assenza di record nei log analizzati da Wazuh e il fallimento dell'iniezione dei record malevoli. In pratica il server è sicuro, ma più che altro perché è sordo.

3.6 Modifica architettura

Una possibile soluzione per continuare la trattazione è passare a un'architettura con **Resolver dedicato**, in questo modo:

- l’attacco potrebbe funzionare
- si ottiene una rappresentazione fedele di come funzionano le reti aziendali moderne

3.6.1 Aggiunta Container e configurazione Resolver

In questa fase, l'infrastruttura è stata evoluta per riflettere un'architettura di rete aziendale realistica, separando le funzioni autoritative da quelle ricorsive.

- **Modifiche al file `docker-compose.yml`:**
  Il server Master è stato rimosso dalla rete pubblica e le porte sono state riassegnate al Resolver per forzare il traffico verso quest'ultimo.

```
# Snippet configurazione Hidden Master
bind9_server:
  networks:
    mgmt_net:
      ipv4_address: 10.0.1.53 # Unica interfaccia attiva
  # ports rimosse per isolamento

# Snippet configurazione Resolver
bind9_resolver:
  networks:
    frontend_net:
      ipv4_address: 172.20.0.53
    mgmt_net:
      ipv4_address: 10.0.1.60
  ports:
    - "53:53/udp"
    - "53:53/tcr"
  volumes:
    - ./resolver/config:/etc/bind
    - ./resolver/log:/var/log/named
```

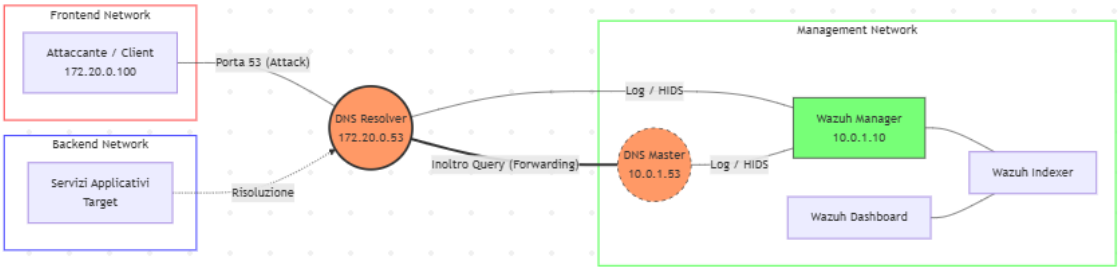
- **Configurazione del Resolver (`named.conf.options`):**
  Il Resolver è stato istruito per inoltrare le richieste relative alla zona locale al Master tramite la rete di gestione e per fare logging.

```
options {
    recursion yes;
    allow-query { any; };
    forwarders { 10.0.1.53; }; // Indirizzo del Master su mgmt_net
    dnssec-validation no;
};
```

Integrazione Wazuh:
Il monitoraggio è stato esteso aggiungendo un nuovo punto di raccolta log per il Resolver nel file `ossec.conf` del Manager, garantendo la visibilità su entrambi i nodi DNS.

> Tutti i test per la validazione del corretto funzionamento sono stati fatti off-documentazione per evitare di mostrare più volte la stessa cosa, in quanto sono processi già documentati e risulterebbero ripetitivi

3.6.2 Infrastruttura di rete aggiornata



Con la modifica architetturale introdotta in questo paragrafo, l'infrastruttura è stata evoluta includendo un Resolver ricorsivo dedicato, separato dal Master autoritativo. Tale configurazione rende teoricamente possibile un attacco di DNS Cache Poisoning di tipo blind spoofing, in quanto il resolver genera query ricorsive verso l'esterno e mantiene uno stato di sessione associato a Transaction ID e porta sorgente randomizzata.

3.7 Preparazione nuovo attacco

A differenza del test descritto nel paragrafo 3.5, dove il server operava esclusivamente come Authoritative Master (e quindi non generava richieste ricorsive per la propria zona), il nuovo scenario introduce una reale superficie di attacco: il resolver, inoltrando le richieste al forwarder (10.0.1.53), diventa potenzialmente vulnerabile a collisioni di Transaction ID e risposte forzate.

L'attacco è stato quindi adattato per:

- Creare tramite Scapy pacchetti "spoofati" con l'IP sorgente del forwarder
- Intercettare il momento esatto in cui il resolver genera una query ricorsiva.
- Inviare una raffica di risposte DNS falsificate con TXID variabili.
- Forzare l'associazione del record A di `www.progetto.ns` all'IP del server clone (172.20.0.80).
- Colpire range di porte UDP randomizzate per superare la Source Port Randomization.

3.7.1 Limiti dello Spoofing in Ambiente Containerizzato

Tuttavia, durante l'esecuzione pratica, sono emerse limitazioni strutturali legate all'ambiente di virtualizzazione utilizzato (Docker Desktop su sistema Windows).

In particolare:

- I pacchetti forgiati tramite Scapy, pur impostando manualmente l'IP sorgente del forwarder (10.0.1.53), venivano riscritti dal gateway virtuale di Docker.
- Il bridge di rete della VM sottostante applicava meccanismi di Source NAT (SNAT), alterando l'header IP prima della consegna al resolver.
- Anche l'utilizzo di tecniche a livello 2 (sendp con frame Ethernet personalizzati) non permetteva di bypassare il layer di astrazione introdotto dall'hypervisor.

Di conseguenza, il resolver riceveva risposte provenienti dall'indirizzo del gateway virtuale (172.20.0.1) anziché dall'indirizzo spoofato, classificandole correttamente come non sollecitate (Unsolicited) e scartandole prima dell'elaborazione applicativa.

Questa limitazione non è imputabile alla robustezza intrinseca del protocollo DNS, bensì al modello di networking containerizzato adottato. Docker Desktop, operando sopra una macchina virtuale (WSL2/Hyper-V), introduce un livello di isolamento che impedisce la reale manipolazione degli header IP a basso livello, rendendo di fatto non replicabile un attacco di blind spoofing puro in tale contesto.

L'esperimento ha quindi evidenziato un aspetto metodologico rilevante: la fedeltà di una simulazione di attacco dipende fortemente dal grado di controllo sul piano di rete sottostante. In ambienti containerizzati non bare-metal, alcune classi di attacco risultano tecnicamente non dimostrabili.

Tuttavia, dal punto di vista dell'attaccante, un ostacolo infrastrutturale non rappresenta un punto di arresto definitivo, bensì uno stimolo a riformulare la strategia. Se lo spoofing diretto del forwarder non è praticabile, occorre analizzare il sistema da un differente angolo di attacco, individuando altre debolezze nel flusso di risoluzione.

In altre parole, l'obiettivo rimane invariato (compromettere l'integrità della cache del resolver), ma il vettore deve adattarsi al contesto operativo.

Pertanto, nelle sezioni successive verrà adottato un approccio alternativo, modellato secondo una logica offensiva più realistica: non tentare di forzare il piano di rete sottostante, bensì sfruttare le dinamiche di fiducia interne al processo di forwarding DNS.

3.8 Cambio di Prospettiva - Abuso di Dynamic DNS Update non Autenticato

Siccome lo spoofing non è possibile, è stato adottato un vettore alternativo fondato sull'abuso di una configurazione errata del Master DNS: l'abilitazione degli aggiornamenti dinamici (Dynamic DNS Update) senza autenticazione.

3.8.1 Modello di Minaccia

Nel protocollo DNS, il meccanismo di Dynamic Update consente la modifica runtime dei record di zona tramite il comando `nsupdate`. In ambienti correttamente configurati, tale operazione è protetta mediante chiavi TSIG (Transaction Signature) o limitata a specifici host autorizzati.

Nel presente scenario, a scopo dimostrativo, sono state introdotte delle misconfigurazioni controllate:

- nel Master DNS (10.0.1.53), abilitando in `named.conf.local`:

  ```
  allow-update { any; };
  allow-transfer { any; };
  ```

- nel Resolver (172.20.0.53), aggiungendo `named.conf.local` le seguenti direttive:

  ```
  zone "progetto.ns" {
      type slave;
      masters { 10.0.1.53; };
      file "/var/cache/bind/db.progetto.ns";
      allow-update-forwarding { any; };
  };
  ```

Tale configurazione consente a qualunque nodo raggiungibile sulla rete di inviare richieste di aggiornamento della zona autoritativa `progetto.ns`, senza autenticazione né verifica crittografica.

Questo scenario simula ambienti legacy o deployment erroneamente configurati, nei quali il controllo degli accessi alle zone non è adeguatamente implementato.

3.8.2 Estensione della visibilità sui log e configurazione IDS

Per garantire la tracciabilità dell'attacco, che in questo scenario non è basato su query malevole (come nel classico cache poisoning) bensì su aggiornamenti dinamici, è stato necessario estendere la visibilità di logging di BIND9 sia sul Master che sul Resolver.

Nel file di configurazione `named.conf.options` dei rispettivi server è stata aggiunta la seguente direttiva: `category update { query_log; };`

Inoltre, limitatamente al nodo Resolver, è stata integrata un'ulteriore direttiva: `category update-security { query_log; };`

Questa configurazione assicura che il demone DNS registri nel file di log ogni singola interazione legata ai tentativi di forwarding degli update (siano essi accettati o rifiutati dal Master). Senza questa modifica propedeutica, l'intera architettura di monitoraggio (stack Wazuh) sarebbe risultata "cieca" rispetto a questo specifico vettore di attacco.

Senza questa piccola modifica lo stack Wazuh avrebbe perso di senso.

Poiché gli aggiornamenti dinamici sono indistinguibili dal traffico legittimo, è stato necessario implementare regole custom (Threat Hunting) sull'HIDS per tracciare le modifiche ai record critici.

Creazione Regola Custom di Threat Hunting in Wazuh

A livello di traffico di rete, gli aggiornamenti dinamici veicolati tramite l'utility `nsupdate` sono strutturalmente indistinguibili dalle normali operazioni legittime. Per questo motivo, è stato necessario implementare una regola *custom* sull'HIDS (Wazuh) per tracciare le modifiche ai record critici.

La regola è stata iniettata direttamente nel container del Wazuh Manager tramite il seguente comando, andando a sovrascrivere il file `local_rules.xml`:

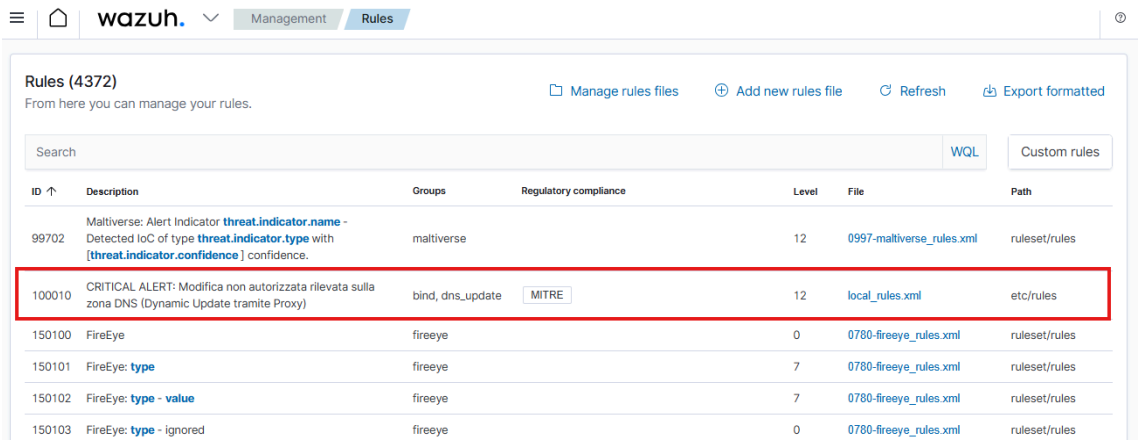
```
<group name="bind, dns_update">
  <rule id="100010" level="12">
    <match>update</match>
    <description>CRITICAL ALERT: Modifica non autorizzata rilevata sulla zona
DNS (Dynamic Update tramite Proxy)</description>
    <mitre>
      <id>T1098</id>
    </mitre>
  </rule>
</group>
```

- **Comando:** `docker exec -it wazuh_manager bash -c "echo -e '<group name=\"bind, dns_update\">\n <rule id=\"100010\" level=\"12\">\n   <match>updating zone</match>\n   <description>CRITICAL ALERT: Modifica non autorizzata rilevata sulla zona DNS (Dynamic Update tramite Proxy)</description>\n   <mitre>\n <id>T1098</id>\n   </mitre>\n </rule>\n</group>' > /var/ossec/etc/rules/local_rules.xml”`

Considerazioni sulla Gestione dei Falsi Positivi

Questa regola garantisce l'innesco di un Allarme Critico (Livello 12) per *qualsiasi* richiesta di update rilevata nei log. Nonostante il vincolo possa apparire poco stringente o soggetto a falsi positivi, questa scelta riflette un approccio *Zero-Trust* applicato in contesti aziendali reali.

Le modifiche ai record DNS autoritativi seguono infatti rigorosi processi di *Change Management*. Quando un amministratore di rete deve effettuare dei cambiamenti legittimi alla zona, l'intervento avviene durante specifiche "finestre di manutenzione" preventivamente comunicate al SOC (Security Operations Center) o ai colleghi responsabili del monitoraggio. Qualsiasi allarme generato da Wazuh al di fuori di queste finestre concordate viene pertanto trattato automaticamente come un indicatore di compromissione reale (*True Positive*), rendendo superflua una regola più granulare.



3.8.3 Esecuzione dell'attacco

L'attacco è stato articolato nelle seguenti fasi logiche:

Fase 1 – Reconnaissance

Dal nodo attaccante (172.20.0.100), è stata effettuata enumerazione del servizio DNS per identificare:

- Record SOA: `docker exec -it kali_attacker dig @172.20.0.53 progetto.ns SOA`

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it kali_attacker dig @172.20.0.53 progetto.ns SOA

; <<>> DiG 9.20.18-1-Debian <<>> @172.20.0.53 progetto.ns SOA
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 29083
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 6b5a56bc411b2cf401000000699b606fa6ddf6144aced2e5 (good)
;; QUESTION SECTION:
;progetto.ns.                   IN      SOA

;; ANSWER SECTION:
progetto.ns.            600427  IN      SOA     ns1.progetto.ns. admin.progetto.ns. 3 604800 86400 2419200 604800

;; Query time: 0 msec
;; SERVER: 172.20.0.53#53(172.20.0.53) (UDP)
;; WHEN: Sun Feb 22 20:00:47 UTC 2026
;; MSG SIZE  rcvd: 114
```

L'output ha confermato l'esistenza della zona, restituendo nella sezione `ANSWER` il server primario `ns1.progetto.ns.`

- Name Server autoritativo: `docker exec -it kali_attacker dig @172.20.0.53 progetto.ns NS`

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it kali_attacker dig @172.20.0.53 progetto.ns NS

; <<>> DiG 9.20.18-1-Debian <<>> @172.20.0.53 progetto.ns NS
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 59085
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: d1796c2d66e52ad301000000699b60ab62ccb8042812bc2e (good)
;; QUESTION SECTION:
;progetto.ns.                    IN      NS

;; ANSWER SECTION:
progetto.ns.            601219  IN      NS      ns1.progetto.ns.

;; ADDITIONAL SECTION:
ns1.progetto.ns.        601219  IN      A       172.20.0.53

;; Query time: 0 msec
;; SERVER: 172.20.0.53#53(172.20.0.53) (UDP)
;; WHEN: Sun Feb 22 20:01:47 UTC 2026
;; MSG SIZE  rcvd: 102
```

Dall'analisi della risposta è stato confermato che `ns1.progetto.ns.` è il server delegato. Un dettaglio tecnico rilevante è emerso dalla sezione `ADDITIONAL`: il resolver ha restituito l'indirizzo `172.20.0.53` per il record A di `ns1.progetto.ns.`. Questo comportamento è coerente con la topologia di rete implementata, in cui il record di zona punta originariamente all'IP pubblico esposto.
Dal punto di vista dell'attaccante esterno, questo nodo rappresenta l'unico punto di contatto (entry-point) per la risoluzione DNS. La strategia offensiva si adatta di conseguenza: si tenterà di veicolare l'attacco di iniezione (*Dynamic Update*) direttamente contro questo indirizzo pubblico, sfruttando il Resolver come vettore intermedio per raggiungere il database della zona.

❗ Non conoscendo a priori l'alberatura della zona, l'attaccante ha proceduto con una fase di brute-forcing DNS basata su dizionario per individuare i record validi. Tramite l'invio massivo di query per i sottodomini più comuni, il resolver ha confermato l'esistenza del record `www.progetto.ns`, il quale rappresenta un target di altissimo valore (presumibilmente l'interfaccia web principale dell'infrastruttura). La prevedibilità della nomenclatura standard (`www`) ha reso l'individuazione del bersaglio immediata.

> La simulazione pratica di una fase di *subdomain enumeration* (brute-forcing) è stata deliberatamente omessa per mantenere l'analisi focalizzata sull'effettiva vulnerabilità architetturale: l'abuso dei DNS Dynamic Updates.

Fase 2 – Verifica Superficie di Attacco

Acquisita la consapevolezza che l'indirizzo `172.20.0.53` rappresenta l'interfaccia esposta per la gestione della zona, l'attaccante procede con la verifica attiva della superficie di attacco. L'obiettivo di questa

fase è accertare se il server accetti aggiornamenti dinamici non autenticati (Dynamic DNS Update) e se sia in grado di inoltrarli al Master Autoritativo nascosto.

Per testare la vulnerabilità in modo non distruttivo, è stato forgiato un payload tramite l'utility `nsupdate`, volto a iniettare un record A fittizio (`test.progetto.ns`) all'interno della zona. Il comando è stato strutturato in un'unica istruzione concatenata per automatizzare l'interazione con il demone:

```
docker exec -it kali_attacker bash -c "echo -e 'server 172.20.0.53\nzone prog
etto.ns\nupdate add test.progetto.ns 60 A 1.1.1.1\nsend' | nsupdate".
```

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it kali_attacker bash -c "echo -e 'server 172.20.0.53\nzone progetto.ns\nupdate add test.progett
o.ns 60 A 1.1.1.1\nsend' | nsupdate"
```

L'assenza di messaggi di errore (come `NOTAUTH` o `REFUSED`) da parte del server ha fornito un primo indicatore positivo circa la fattibilità dell'attacco.

Per ottenere la conferma definitiva dell'avvenuta compromissione dell'integrità della zona, è stata eseguita una query di verifica mirata al record appena iniettato:

```
docker exec -it kali_attacker dig @172.20.0.53 test.progetto.ns +short
```

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it kali_attacker dig @172.20.0.53 test.progetto.ns +short
1.1.1.1
```

La risoluzione corretta del record `test.progetto.ns` nell'indirizzo IP `1.1.1.1` dimostra inequivocabilmente la presenza della vulnerabilità. L'esperimento conferma che l'architettura è soggetta a un difetto critico nella gestione del trust: il server esposto funge da proxy non autenticato, accettando modifiche arbitrarie esterne e inoltrandole al Master Autoritativo, il quale le applica senza ulteriore validazione crittografica. La catena di fiducia logica risulta pertanto compromessa.

Fase 3 – Exploitation

Confermata la vulnerabilità, l'attaccante ha via libera per eseguire l'aggiornamento malevolo del record target.

Allestimento War Room

Al fine di acquisire le evidenze forensi necessarie per la successiva fase di analisi, è stata predisposta una War Room per garantire una visibilità a 360° sull'infrastruttura durante l'iniezione del payload:

- **Terminale A:** `docker exec bind9_resolver tcpdump -i any port 53 -w /var/log/named/update_attack.pcap`

  Dedicato all'acquisizione del traffico grezzo sul nodo esposto (Resolver)

- **Terminale B:** `docker exec bind9_server tail -f /var/log/named/query.log`

  Dedicato all'ispezione real-time dei log di servizio di BIND9

- **Terminale C:** `docker exec -it kali_attacker bash -c "echo -e 'server 172.20.0.53\nzone progetto.ns\nupdate delete www.progetto.ns A\nupdate add www.progetto.ns 60 A 172.20.0.80\nsend' | nsupdate"`

Riservato al nodo attaccante per sganciare il payload malevolo. Il comando esegue in sequenza l'eliminazione del record legittimo e l'inserimento del record clone (puntante all'IP 172.20.0.80 controllato dall'attaccante).

Propagazione ed Evidenze

Una volta lanciati questi comandi, il Terminale B ha mostrato a video il seguente output, confermando la ricezione e l'elaborazione dell'update:

```
22-Feb-2026 21:35:20.465 update: info: client @0x7f249ca3d000 10.0.1.60#4420
1: updating zone 'progetto.ns/IN': deleting rrset at 'www.progetto.ns' A
22-Feb-2026 21:35:20.465 update: info: client @0x7f249ca3d000 10.0.1.60#4420
1: updating zone 'progetto.ns/IN': adding an RR at 'www.progetto.ns' A 172.2
0.0.80
```

Per forzare la scrittura delle modifiche e velocizzare la propagazione degli aggiornamenti, è stato necessario intervenire manualmente sul demone BIND9. Sono stati eseguiti i seguenti comandi:

```
#Per far aggiornare subito il file di zona tramite il .jnl
#(file journal generato all'update)
docker exec bind9_server rndc sync -clean
#Riavvio resolver per svuotare cache
docker compose restart bind9_resolver
```

Verifica dell'Attacco

Ciò che abbiamo visto fin'ora (l'output del terminale B) indica che la manipolazione della zona è avvenuta con successo sulla carta. Tuttavia, per avere la verifica effettiva della riuscita dell'attacco (DNS Poisoning), è stata effettuata un'interrogazione tramite l'utility `dig` dal Terminale C.

Per garantire che la risoluzione del nome non fosse falsata da vecchi dati in memoria e simulare lo scenario peggiore per un nuovo client, **questa query di verifica è stata eseguita solo dopo aver eliminato e rimpiazzato il file del resolver**, forzando così una risoluzione "pulita" che interrogasse direttamente la zona appena alterata: `docker exec bind9_resolver rm -f /var/cache/bind/db.progetto.ns`

La verifica effettiva viene effettuata tramite l'utility `dig` lanciata dal terminale C: `docker exec -it kali_attacker dig @172.20.0.53 www.progetto.ns`

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it kali_attacker
 dig @172.20.0.53 www.progetto.ns +short
172.20.0.80
```

L'esito positivo dell'interrogazione DNS conferma che il record A per `www.progetto.ns` è stato ufficialmente compromesso e sovrascritto con l'IP controllato dall'attaccante (`172.20.0.80`).

Questa verifica sancisce la conclusione della fase di *Exploitation*: l'attaccante è riuscito ad aggirare le restrizioni che impedivano il DNS Spoofing diretto, sfruttando invece una vulnerabilità logica legata all'assenza di controlli crittografici sui DNS Dynamic Updates. La manipolazione del record autoritativo sul Master Server ha di fatto innescato una compromissione a cascata.

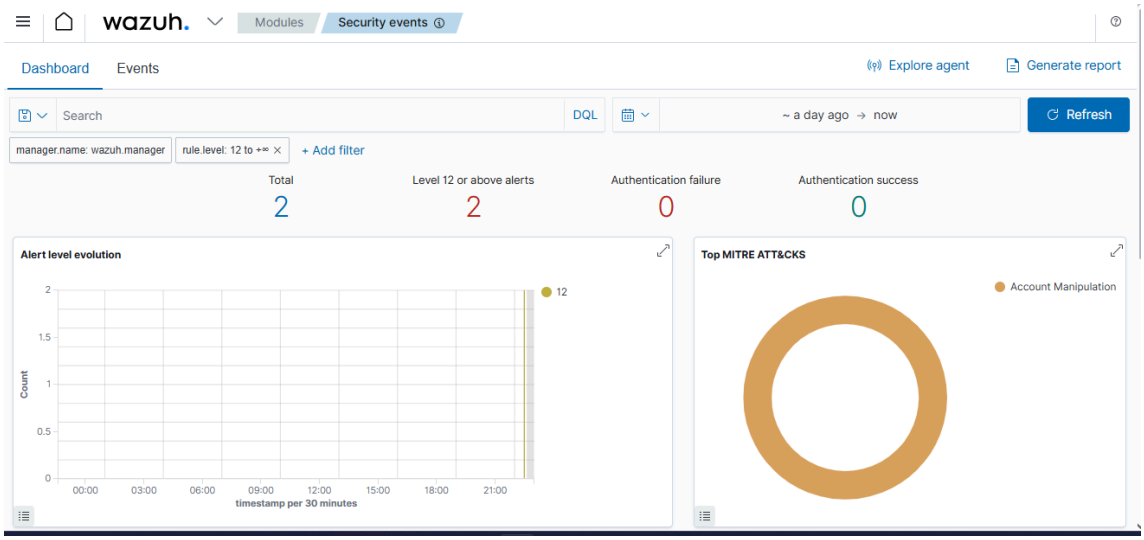
Nel prossimo capitolo verrà analizzato nel dettaglio l'impatto di questa operazione sull'infrastruttura di risoluzione (in particolare sull'avvelenamento della cache del Resolver) e verranno tratte le considerazioni architetturali relative a questa classe di attacchi.

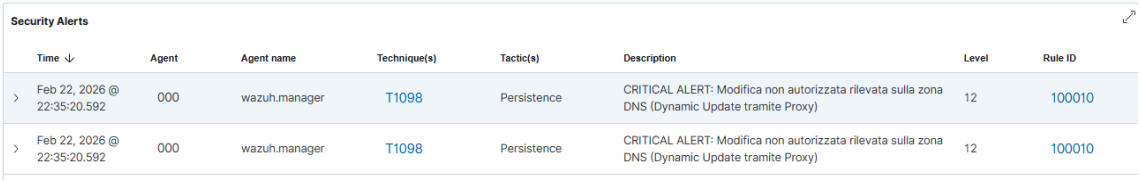
Capitolo 4: Detection & Analysis

4.1 Rilevamento dell'Anomalia tramite HIDS (Wazuh)

Grazie alla configurazione preventiva del logging di BIND9 e all'implementazione della regola custom sul Wazuh Manager (descritta nel capitolo precedente), l'attacco di iniezione tramite Dynamic Update non è passato inosservato.

Non appena il payload malevolo è stato sganciato dal nodo attaccante, l'agente Wazuh ha intercettato la scrittura nei log del server BIND9, riconoscendo il pattern malevolo associato alla modifica non autorizzata della zona (`updating zone`). Questo ha innescato la regola ID `100010`, generando un *Critical Alert* (Livello 12) mappato sulla tattica MITRE ATT&CK T1098 (*Account Manipulation* / manipolazione degli asset di fiducia).





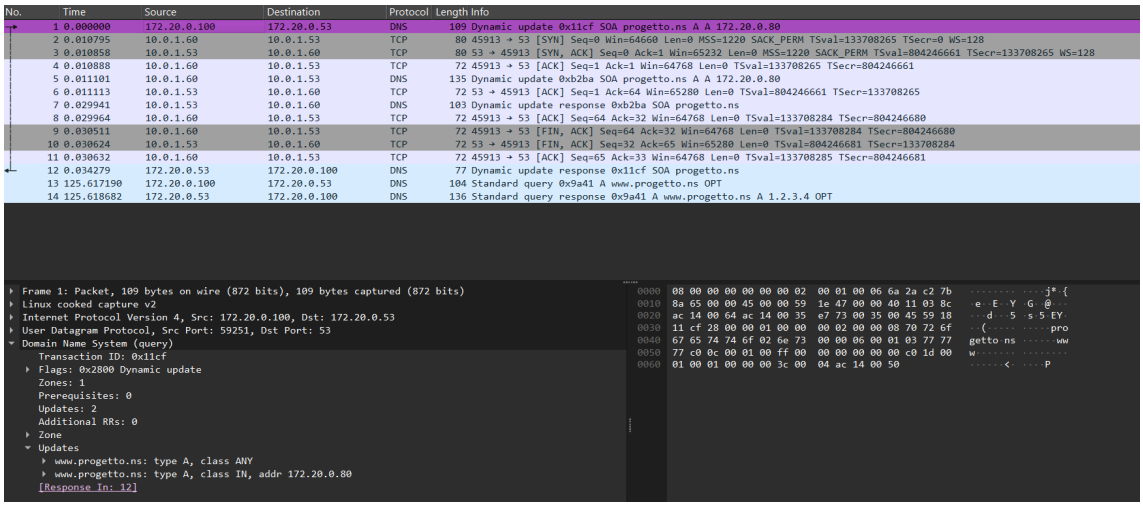
Security Alerts

Time	Agent	Agent name	Technique(s)	Tactic(s)	Description	Level	Rule ID
Feb 22, 2026 @ 22:35:20.592	000	wazuh.manager	T1098	Persistence	CRITICAL ALERT: Modifica non autorizzata rilevata sulla zona DNS (Dynamic Update tramite Proxy)	12	100010
Feb 22, 2026 @ 22:35:20.592	000	wazuh.manager	T1098	Persistence	CRITICAL ALERT: Modifica non autorizzata rilevata sulla zona DNS (Dynamic Update tramite Proxy)	12	100010

Questo evento rappresenta il punto di innesco per le procedure di *Incident Response*, confermando che, sebbene l'attacco a livello DNS sia andato a buon fine, il sistema di monitoraggio ha fornito la visibilità necessaria per identificare la compromissione in tempo reale.

4.2 Analisi Forense del Traffico di Rete (PCAP)

A complemento degli alert generati dall'HIDS, è stata condotta un'analisi forense sul traffico di rete grezzo catturato durante la fase di *Exploitation*. Il file `update_attack.pcap`, generato dal demone `tcpdump` sul nodo Resolver (Terminale A della War Room), è stato ispezionato per ricostruire l'esatta dinamica a livello di protocollo.



Dall'analisi della cattura emergono tre fasi di comunicazione distinte che descrivono l'intero ciclo di vita dell'attacco:

1. **Iniezione del Payload (Pacchetto 1):**

   L'attaccante (`172.20.0.100`) invia un pacchetto UDP alla porta 53 dell'interfaccia esposta del Resolver (`172.20.0.53`). L'ispezione del payload (riquadro inferiore dello screenshot) rivela i dettagli dell'exploit:

   - **Flags:** Il pacchetto è marcato esplicitamente come `0x2800 Dynamic update`, confermando che non si tratta di una query standard.
   - **Updates (2):** Questa sezione contiene l'effettivo "veleno". Si notano chiaramente le due istruzioni sequenziali forgiate tramite `nsupdate`: la prima elimina il record preesistente (`type A, class ANY`), mentre la seconda inserisce il record clone (`type A, class IN, addr 172.20.0.80`).
   - **Additional RRs (0):** È l'evidenza forense più critica. Il valore zero indica l'assenza totale di record aggiuntivi, dimostrando matematicamente che la richiesta è **priva di qualsiasi firma crittografica TSIG**. Il pacchetto è stato accettato dal sistema nonostante fosse completamente in chiaro e non autenticato.

2. **Forwarding Interno via TCP (Pacchetti 2 - 11):**
   Questa è la fase più critica che dimostra la vulnerabilità architetturale. Il Resolver, non essendo autoritativo per la zona ma avendo una direttiva di *forwarding* per gli update, agisce da ponte (bridge) verso la rete di backend.

   - **Pacchetti 2-4:** Il Resolver (tramite il suo IP interno `10.0.1.60`) instaura un classico *TCP Three-way Handshake* con il Master server nascosto (`10.0.1.53`). BIND9 utilizza di default il protocollo TCP per inoltrare gli aggiornamenti di zona per garantirne l'affidabilità.

- **Pacchetto 5:** Il payload malevolo viene inoltrato dal Resolver al Master all'interno della sessione TCP appena creata.
- **Pacchetto 7:** Il Master, fidandosi implicitamente del nodo Resolver della sua stessa sottorete, accetta la modifica e risponde con un **Dynamic update response** di successo (`0xb2ba`).
- **Pacchetti 8-11:** La connessione TCP tra i due server viene chiusa regolarmente (*Teardown* con i flag FIN/ACK).

3. **Conferma all'Attaccante (Pacchetto 12):**
   Avendo ricevuto esito positivo dal Master, il nodo esposto `172.20.0.53` chiude la transazione inviando a sua volta un **Dynamic update response** (sempre riferito alla transazione originale `0x11cf`) all'IP dell'attaccante `172.20.0.100`.

La registrazione di questa sequenza documenta come un attaccante esterno sia riuscito a far propagare un pacchetto non autenticato fino al cuore dell'infrastruttura (il server Master), semplicemente sfruttando la relazione di fiducia esistente tra le interfacce interne dei container.

4.3 Impatto sulla Cache del Resolver

Dal punto di vista dell'infrastruttura di risoluzione, l'attacco ha prodotto conseguenze devastanti sulla catena di fiducia dell'infrastruttura. Dopo aver eseguito il flush della cache del Resolver ricorsivo, una nuova query per `www.progetto.ns` ha restituito l'indirizzo `172.20.0.80` (il server clone controllato dall'attaccante).

L'avvelenamento della cache è avvenuto in modo strutturalmente perfetto dal punto di vista del protocollo DNS. L'aggiornamento malevolo presentava infatti tutte le caratteristiche di una transazione legittima:

- **Transaction ID valido**
- **Porta corretta**
- **Provenienza legittima** (il forwarder configurato)
- **Nessuna anomalia sintattica**

Questo dimostra che l'attacco non ha sfruttato debolezze probabilistiche o crittografiche del protocollo DNS (come nel tradizionale attacco di Cache Poisoning), bensì una relazione di fiducia mal configurata tra i nodi dell'architettura.

4.4 Considerazioni Architetturali

L'esperimento dimostra in modo inequivocabile che la resilienza contro attacchi di spoofing non equivale a protezione contro le compromissioni logiche della catena di fiducia.

In assenza di DNSSEC per la validazione crittografica delle risposte e di controlli rigorosi (come le firme TSIG o ACL restrittive) sugli aggiornamenti dinamici, l'integrità dell'intera zona dipende esclusivamente dalla corretta configurazione del Master. Il Resolver, agendo da proxy non autenticato, ha di fatto esteso la superficie d'attacco del Master fino alla rete esposta.

L'approccio adottato riflette una dinamica offensiva realistica e sofisticata: quando un vettore diretto (come il cache poisoning blind) risulta impraticabile o mitigato, l'attaccante ricerca e sfrutta configurazioni deboli o meccanismi di fiducia impliciti all'interno dell'infrastruttura stessa.

Capitolo 5: Hardening

Prima di entrare nel vivo di questa fase, c'è bisogno di effettuare un'operazione di *Recovery*, per poi passare alla *Mitigation* vera e propria che impedirà ulteriori scenari futuri come quello presentatosi durante la simulazione.

5.1 Eradication & Recovery

Per prima cosa dobbiamo ripristinare il file di zona al suo stato originale. È buona norma (e best practice aziendale) mantenere sempre dei backup in modo che il ripristino (*Roll-Back*) sia agevole e non richieda di ripetere tutte le procedure di configurazione manualmente. I passi per riportare il sistema allo stato non corrotto sono i seguenti:

Fase 1 - Congelamento Zona sul Master

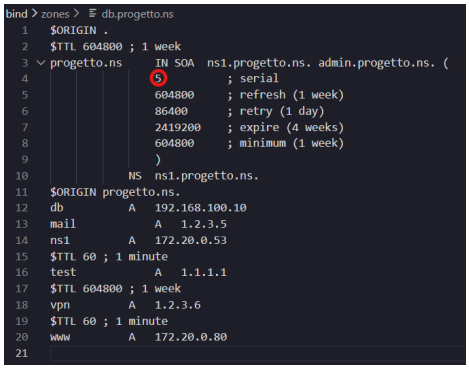
Bisogna bloccare temporaneamente l'accettazione di nuovi aggiornamenti dinamici e forzare BIND9 a scrivere tutte le modifiche pendenti dal file journal (`.jnl`) ,se presenti, al file di zona testuale.

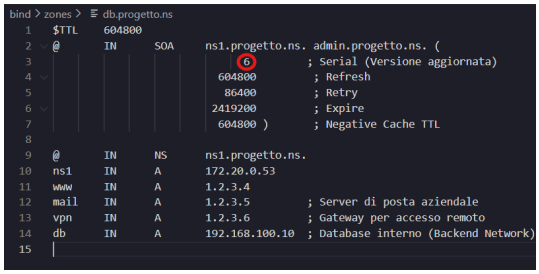
```
docker exec bind9_server rndc freeze progetto.ns
```

Fase 2 - Sostituzione del file di zona

Ora è possibile sostituire il file di zona `db.progetto.ns` corrotto con quello precedente all'attacco, la cosa più importante è però aggiornare il seriale.

Quindi per prima cosa bisogna controllare il seriale attuale e poi incrementarlo nel nuovo file.





Come si denota dalle immagini è stato incrementato il seriale da 5 a 6.

Fase 3 - Scongelamento e reload della zona

Una volta salvato il file, bisogna sbloccare la zona. In questo modo BIND9 leggerà il nuovo file, vedrà che il seriale è aumentato, cancellerà il vecchio journal (se presente) e applicherà la tua configurazione pulita.

```
docker exec bind9_server rndc thaw progetto.ns
```

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec bind9_server rndc
 thaw progetto.ns
A zone reload and thaw was started.
```

Fase 4 - Flush della cache del Resolver

Infine, dobbiamo assicurarci che il Resolver dimentichi l'IP malevolo che si era salvato in memoria.

```
docker exec bind9_resolver rm -f /var/cache/bind/db.progetto.ns
```

(riavviamo anche il container per sicurezza `docker compose restart bind9_resolver`)

Fase 5 - Verifica della pulizia

Il nome della fase è autoesplicativo: per assicurarci che tutto sia tornato alla normalità bisogna testare il resolver con una query di controllo

```
docker exec -it kali_attacker dig @172.20.0.53 www.progetto.ns +short
```

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it kali_attacker
 dig @172.20.0.53 www.progetto.ns +short
1.2.3.4
```

È stato scelto il container dell'attaccante perché nell'architettura attuale rappresenta l'esterno. Poiché l'obiettivo di questa fase è pulire il sistema, è fondamentale verificare che il mondo esterno ricominci a vedere l'IP legittimo e non più quello del sito clone.

Ora che la fase di *Recovery* è terminata con successo, è possibile procedere con la *Mitigation*, ovvero l'hardening del sistema.

5.2 Hardening Diretto (TSIG e ACL)

L'analisi dell'attacco condotto nel Capitolo 3 ha evidenziato una vulnerabilità architetturale critica: il modello di fiducia (*Trust Model*) basato esclusivamente sugli indirizzi IP. Il Master Server accettava le modifiche alla zona fidandosi implicitamente del fatto che la richiesta provenisse dall'IP del Resolver interno, ignorando l'assenza di un'autenticazione a livello applicativo.

Per mitigare questa falla in modo definitivo, si è optato per l'implementazione del protocollo **TSIG (Transaction Signature)**, descritto nella RFC 2845. TSIG introduce un approccio *Zero-Trust*, vincolando le transazioni DNS (come i *Dynamic Updates* e i *Zone Transfers*) al possesso di una chiave crittografica simmetrica condivisa (HMAC).

L'implementazione è stata strutturata in tre fasi logiche, seguendo il principio della *Defense-in-Depth* (Difesa in Profondità).

Fase 1 - Generazione del Segreto Crittografico

La prima operazione è consistita nella generazione della chiave crittografica direttamente all'interno dell'ambiente BIND9. È stato impiegato l'algoritmo di hashing forte HMAC-SHA256 tramite l'utility di sistema `tsig-keygen`:

```
docker exec -it bind9_server tsig-keygen -a HMAC-SHA256 tsig-update
```

Il comando ha generato il seguente blocco di configurazione, contenente il segreto codificato in Base64:

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it bind9_server tsig-keygen -a HMAC-SHA256 tsig-update
key "tsig-update" {
        algorithm hmac-sha256;
        secret "CxzmZ0nnbzWXnxGjcCuDycgJ6vIplu2Ammo4rEVSO1k=";
};
```

Fase 2 - Messa in Sicurezza del Server Master (Backend)

Il blocco contenente la chiave è stato integrato nel file di configurazione `named.conf.local` del Master Server. Successivamente, le Access Control List (ACL) della zona autoritativa `progetto.ns` sono state ristrette per esigere la firma crittografica per due direttive fondamentali:

1. `allow-update`: Protezione dell'Integrità. Impedisce la scrittura non autorizzata di record.
2. `allow-transfer`: Protezione della Riservatezza. Impedisce l'esfiltrazione della mappa di rete tramite richieste AXFR/IXFR.

La configurazione della zona sul Master è stata così modificata:

```
bind > config > named.conf.local
key "tsig-update" {
        algorithm hmac-sha256;
        secret "CxzmZ0nnbzWXnxGjcCuDycgJ6vIplu2Ammo4rEVSO1k=";
};

zone "progetto.ns" {
    type master;
    file "/var/lib/bind/db.progetto.ns";
    allow-query { any; };
    allow-update { key "tsig-update"; };
    allow-transfer { key "tsig-update"; };
};
```

Fase 3 - Hardening del Resolver e Defense-in-Depth

Affinché il Resolver potesse continuare a operare legittimamente come server *Slave* e aggiornare la propria cache, è stato necessario istruirlo a utilizzare la chiave TSIG per autenticarsi presso il Master durante le richieste di trasferimento di zona.

Inoltre, in ottica di *Defense-in-Depth*, si è deciso di non limitarsi a bloccare gli attacchi sul backend, ma di arrestarli direttamente all'ingresso della rete (Edge). Modificando la direttiva `allow-update-forwarding`, il Resolver è stato configurato per scartare istantaneamente qualsiasi pacchetto di aggiornamento

sprovvisto di firma, prevenendo potenziali attacchi DoS (Denial of Service) mirati a saturare la connessione TCP tra Resolver e Master.

Il file `named.conf.local` del Resolver è stato così strutturato (previa dichiarazione della chiave TSIG):

```
resolver > config > named.conf.local
key "tsig-update" {
        algorithm hmac-sha256;
        secret "CxzmZ0nnbzWXnxGjcCuDycgJ6vIplu2Ammo4rEVSO1k=";
};


zone "progetto.ns" {
    type slave;
    masters { 10.0.1.53 key "tsig-update"; };
    file "/var/cache/bind/db.progetto.ns";
    allow-update-forwarding { key "tsig-update"; };
};

```

Fase 4 - Validazione delle Contromisure (Test di Sicurezza)

A valle dei riavvii dei servizi BIND9 (`docker compose restart bind9_server bind9_resolver`), la solidità dell'infrastruttura è stata validata riproducendo gli scenari d'attacco e verificando l'operatività legittima (Red Team vs Blue Team).

> 💡 **La scelta di condurre i test dal container `kali_attacker` non è casuale:** nell'architettura simulata, questo nodo rappresenta l'esterno dell'infrastruttura (un host su Internet o un client in una rete non *trusted*). Effettuare i test da questa prospettiva permette di verificare empiricamente il perimetro esposto, accertandosi che i blocchi di sicurezza funzionino per le connessioni in ingresso e che il "mondo esterno" interagisca correttamente con le policy appena implementate.

Test 1: Tentativo di Spoofing (Attaccante Esterno)
Replicando l'esatto vettore di attacco utilizzato nel Capitolo 3 (assenza di chiave), il sistema ha respinto la richiesta alla frontiera:

- **comando:** `docker exec -it kali_attacker bash -c "echo -e 'server 172.20.0.53\nzone progetto.ns\nupdate add pwned.progetto.ns 60 A 6.6.6.6\nsend' | nsupdate"`
- **output:**

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it kali_attacker bash -c "echo -e 'server 172.20.0.53\nzone progetto.ns\nupdate add pwned.progetto.ns 60 A 6.6.6.6\nsend' | nsupdate"
update failed: REFUSED
```

Test 2: Tentativo di Zone Transfer (Attaccante Esterno)

L'attaccante ha tentato di enumerare la zona tramite una query AXFR per mappare l'infrastruttura, scontrandosi con i nuovi vincoli di riservatezza:

- **comando:** `docker exec -it kali_attacker dig @172.20.0.53 progetto.ns AXFR`
- **output:**

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it kali_attacker dig @172.20.0.53 progetto.ns AXFR

; <<>> DiG 9.20.18-1-Debian <<>> @172.20.0.53 progetto.ns AXFR
; (1 server found)
;; global options: +cmd
; Transfer failed.
```

Test 3: Aggiornamento Dinamico Autorizzato (Amministratore Legittimo)

Infine, è stato simulato un intervento amministrativo autorizzato, iniettando un nuovo record (`admin.progetto.ns` → `10.10.10.10`) fornendo la firma crittografica tramite il parametro -y dell'utility `nsupdate`:

- **comando:** `docker exec -it kali_attacker bash -c "echo -e 'server 172.20.0.53\nzone progetto.ns\nupdate add admin.progetto.ns 60 A 10.10.10.10\nsend' | nsupdate -y hmac-sha256:tsig-update:[CHIAVE_BASE64]==”`
- **output:**

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it kali_attacker bash -c "echo -e 'server 172.20.0.53\nzone progetto.ns\nupdate add admin.proget
to.ns 60 A 10.10.10.10\nsend' | nsupdate -y hmac-sha256:tsig-update:CxzmZ0nnbzWXnxGjcCuDycgJ6vIplu2Ammo4rEVSO1k="
```

Dopo aver catalizzato l’aggiornamento del journal del Master e pulito la cache del Resolver tramite:

```
docker exec bind9_server rndc sync -clean
docker exec bind9_resolver rm -f /var/cache/bind/db.progetto.ns
```

Per validare l’avvenuta creazione del record e dimostrare che l'implementazione di sicurezza non ha compromesso l'operatività del personale autorizzato è stata fatta un’interrogazione tramite `dig`:

`docker exec -it kali_attacker dig @172.20.0.53 admin.progetto.ns +short`

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it kali_attacker dig @172.20.0.53 admin.progetto.ns +short
10.10.10.10
```

Visibilità e Monitoraggio (Integrazione HIDS e Audit Trail)

L'hardening preventivo si integra perfettamente con le logiche di *Detection* implementate nei capitoli precedenti tramite l'HIDS Wazuh.
Durante l'esecuzione del Test 3 (l'aggiornamento legittimo firmato dall'amministratore), il demone BIND9 ha registrato l'avvenuta transazione. Come si evince dall'immagine seguente, l'agente Wazuh ha immediatamente intercettato l'evento, innescando la regola custom `100010` e generando un alert di livello critico sulla dashboard del SIEM.

Security Alerts

Time	Agent	Agent name	Technique(s)	Tactic(s)	Description	Level	Rule ID
Feb 23, 2026 @ 15:43:27.277	000	wazuh.manager	T1098	Persistence	CRITICAL ALERT: Modifica non autorizzata rilevata sulla zona DNS (Dynamic Update tramite Proxy)	12	100010
Feb 23, 2026 @ 15:43:27.277	000	wazuh.manager	T1098	Persistence	CRITICAL ALERT: Modifica non autorizzata rilevata sulla zona DNS (Dynamic Update tramite Proxy)	12	100010

Questo comportamento del SIEM è di fondamentale importanza architetturale. La regola originaria era stata scritta per rilevare *qualsiasi* tentativo di modifica passante per il nodo proxy. Ora che l'infrastruttura è protetta da TSIG (Prevention), questo alert diventa non solo un allarme di compromissione ma anche un prezioso strumento di **Audit Trail**. Anche se l'operazione è crittograficamente legittima e autorizzata, il Security Operations Center (SOC) riceve comunque una notifica tempestiva di ogni singola modifica strutturale apportata alla zona DNS. Questo dimostra una perfetta sinergia tra i controlli di sicurezza preventivi (che bloccano gli attaccanti, come verificato nei Test 1 e 2) e i controlli investigativi (che tracciano le operazioni degli amministratori).

Conclusioni sull'Hardening Diretto

L'implementazione delle firme TSIG e l'irrigidimento delle ACL tramite vincoli crittografici hanno permesso di chiudere definitivamente la vulnerabilità di iniezione via update dinamico. Si è passati da un modello di sicurezza debole a un'architettura Zero-Trust solida. Attualmente, l'infrastruttura garantisce l'integrità dei dati e la riservatezza della mappa di rete contro attori esterni, pur mantenendo intatta la scalabilità operativa per gli amministratori. Questo risultato costituisce una base sicura (*protezione server-side*) su cui andare a innestare la successiva e ultima fase di hardening a tutela dei client: DNSSEC.

5.3 Hardening Strutturale (DNSSEC)

Mentre il protocollo TSIG protegge l'integrità delle transazioni tra server, l'implementazione di **DNSSEC (Domain Name System Security Extensions)** estende la sicurezza ai client finali. DNSSEC garantisce l'origine e l'integrità dei dati DNS attraverso la firma crittografica dei record, prevenendo attacchi di *Cache Poisoning* e *Man-in-the-Middle*.

Il ruolo delle chiavi crittografiche

Il funzionamento di DNSSEC si basa su una coppia di chiavi asimmetriche (pubblica e privata). Nella prassi standard, la sicurezza di una zona è garantita dalla cooperazione di due tipologie di chiavi, ognuna con un ruolo specifico:

- **Zone Signing Key (ZSK - Flag 256)**: È la chiave utilizzata per firmare quotidianamente i record della zona (A, MX, CNAME). È pensata per essere ruotata frequentemente e per avere una lunghezza tale da bilanciare sicurezza e velocità di calcolo.
- **Key Signing Key (KSK - Flag 257)**: È la chiave "madre" che firma esclusivamente il record contenente le chiavi stesse (DNSKEY). Agisce come punto di ancoraggio per la fiducia; la sua parte pubblica viene idealmente depositata presso il registrar (nel record DS della zona superiore) per estendere la catena di fiducia verso l'alto.

Negli standard più moderni, queste due funzioni possono essere accorpate in una **Combined Signing Key (CSK)**, che semplifica la gestione mantenendo i medesimi requisiti di sicurezza. L'introduzione di queste estensioni trasforma il DNS da un sistema basato sulla "fiducia implicita" a un sistema basato sulla **verifica crittografica esplicita**, dove ogni risposta contiene le prove matematiche della propria autenticità.

> Per questo progetto si è scelta un'implementazione di ultima generazione basata su **Fully Automated DNSSEC**, disponibile nelle versioni più recenti di BIND9 (9.20+). Questo approccio

> sposta la gestione del ciclo di vita delle chiavi dall'amministratore al demone stesso, garantendo maggiore affidabilità e riducendo il rischio di errori umani.

5.3.1 Architettura della Firma (Master Side)

Sul Master Server, la configurazione è stata progettata per operare in modalità **Inline Signing**. In questa modalità, BIND mantiene intatto il file di zona originale e genera dinamicamente una versione firmata (`.signed`) in memoria e su disco.

Le scelte tecniche implementate nel file `named.conf.local` includono:

- `dnssec-policy default`: Invece di gestire manualmente chiavi RSA (ZSK/KSK), è stata adottata la policy di default moderna che utilizza l'algoritmo **ECDSA P-256** (Curve Ellittiche). Questo garantisce firme più brevi, minor consumo di CPU e una sicurezza superiore rispetto ai vecchi standard.
- **Gestione Automatica**: BIND genera autonomamente le chiavi nella directory `/etc/bind/keys/` e gestisce la loro rotazione temporale.

bind > config > named.conf.local

```
key "tsig-update" {
        algorithm hmac-sha256;
        secret "CxzmZ0nnbzWXnxGjcCuDycgJ6vIplu2Ammo4rEVSO1k=";
};

zone "progetto.ns" {
    type master;
    file "/var/lib/bind/db.progetto.ns";
    allow-query { any; };

    allow-update { key "tsig-update"; };
    allow-transfer { key "tsig-update"; };

    key-directory "/etc/bind/keys";
    inline-signing yes;
    dnssec-policy default;
};
```

Invece per quanto riguarda il file `db.progetto.ns`:

- **Vincoli di Sicurezza (TTL)**: Per garantire la validità delle firme, i tempi di cache (TTL) della zona sono stati uniformati a **86400 secondi (1 giorno)**, impedendo che i client memorizzino record con firme crittografiche scadute.

5.3.2 Configurazione della Validazione (Master e Resolver)

Per garantire la massima coerenza dell'infrastruttura, la direttiva `dnssec-validation auto;` è stata abilitata nel file `named.conf.options` di **entrambi i nodi** (Master e Resolver).

L'uso del parametro `auto` istruisce i demoni a utilizzare le *Root Trust Anchors* aggiornate e integrate nel software, permettendo ai server di verificare non solo le zone interne, ma anche l'integrità di qualsiasi risposta proveniente dall'Internet pubblica.

5.3.3 Gestione dell'eccezione locale (Resolver Side)

Il Resolver, agendo come punto di contatto per i client in un ambiente di laboratorio con un Top-Level Domain (TLD) non registrato (`.ns`), richiedeva una configurazione specifica per evitare fallimenti di validazione (SERVFAIL). Non esistendo una catena di fiducia verso i Root Server ufficiali per il suffisso `.ns`, è stata implementata una **Validation Exception**:

```
dnssec-validation auto;
validate-except { "progetto.ns"; };
```

resolver > config > named.conf.options

```
options {
    directory "/var/cache/bind";
    recursion yes;
    allow-query { any; };
    forwarders {
        10.0.1.53;
    };
    dnssec-validation auto;
    validate-except { "progetto.ns"; };
};
```

Questa strategia permette al Resolver di mantenere attiva la protezione DNSSEC globale, trattando al contempo `progetto.ns` come una "isola di fiducia" locale i cui record vengono accettati direttamente dal Master autoritativo.

5.3.4 Verifica dell'Integrità Crittografica

L'avvenuta messa in sicurezza è stata verificata tramite l'utility `dig` dal nodo `kali_attacker`.
L'interrogazione dei record di tipo **DNSKEY** ha confermato l'esposizione della chiave pubblica:

```
docker exec -it kali_attacker dig @172.20.0.53 progetto.ns DNSKEY +multiline
```

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it kali_attacker dig @172.20.0.53 progetto.ns DNSKEY +multiline

; <<>> DiG 9.20.18-1-Debian <<>> @172.20.0.53 progetto.ns DNSKEY +multiline
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 13688
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: f7dcd5122c747d1e01000000699cb9a790884ef0781f1ae2 (good)
;; QUESTION SECTION:
;progetto.ns.           IN DNSKEY

;; ANSWER SECTION:
progetto.ns.            3600 IN DNSKEY 257 3 13 (
                                M4H6M3OhmWh9GmzjzUAT04difWIubFO+9wNCPI8mfAk/
                                EonW3J895kcJ4Im/n5mpst1GnLi3OONS1eRE1r1CxA==
                                ) ; KSK; alg = ECDSAP256SHA256 ; key id = 30011

;; Query time: 10 msec
;; SERVER: 172.20.0.53#53(172.20.0.53) (UDP)
;; WHEN: Mon Feb 23 20:33:43 UTC 2026
;; MSG SIZE  rcvd: 148
```

Risultati del test:

- **Status: NOERROR**: Il Resolver ha validato correttamente la richiesta.
- **Flag 257 (CSK)**: È stata rilevata una *Combined Signing Key*, che unisce le funzioni di ZSK e KSK per una gestione semplificata.
- **Algoritmo 13**: Conferma l'utilizzo dello standard ECDSA P-256 con SHA-256.

Questa implementazione conclude l'hardening dell'infrastruttura, garantendo che ogni risposta DNS ricevuta dai client sia verificabile e non manipolabile da attori malevoli.

5.4 Risultati Ottenuti

La fase di hardening descritta in questo capitolo ha permesso di risolvere le vulnerabilità strutturali identificate durante la fase di analisi. L'approccio adottato non si è limitato alla risoluzione dei singoli problemi, ma ha puntato alla creazione di un'architettura di difesa a più livelli (Defense in Depth).

I risultati ottenuti possono essere riassunti in tre pilastri fondamentali:

1. **Protezione dell'Infrastruttura (TSIG)**: L'implementazione delle chiavi TSIG ha blindato le comunicazioni tra Master e Resolver. Il trasferimento di zona (AXFR), precedentemente esposto a rischi di *zone walking* o manipolazione, è ora un processo autenticato e autorizzato, garantendo che solo i nodi fidati possano scambiarsi i database dei record.
2. **Integrità del Dato (DNSSEC)**: Grazie all'adozione del protocollo DNSSEC con firme ECDSA, l'origine di ogni risposta DNS è ora matematicamente certa. Questo neutralizza definitivamente la minaccia del *Cache Poisoning*, poiché il Resolver è in grado di scartare qualsiasi risposta non firmata o la cui firma risulti alterata.
3. **Automazione**: L'utilizzo di BIND 9.20 e delle policy di firma automatica ha permesso di implementare standard di sicurezza d'avanguardia (come l'Algoritmo 13 e le chiavi CSK) riducendo l'onere amministrativo e i rischi derivanti dalla gestione manuale delle chiavi crittografiche.

In conclusione, l'infrastruttura "progetto.ns" è passata da uno stato di fiducia implicita a un modello **Zero Trust** a livello DNS. Ogni transazione è autenticata e ogni dato è firmato, rendendo la rete resiliente ai più comuni vettori di attacco diretti al servizio di risoluzione dei nomi.

Capitolo 6: Validation & Analysis

Questo capitolo è dedicato alla fase di collaudo e validazione (*Verification & Validation*) dell'infrastruttura post-hardening. Attraverso una serie di test pratici e l'analisi forense dei pacchetti di rete, verrà dimostrata l'efficacia delle contromisure crittografiche (TSIG e DNSSEC) implementate nel capitolo precedente, confrontando il comportamento attuale del sistema con le vulnerabilità rilevate durante le fasi di attacco.

6.1 Fase 1: Neutralizzazione dell'Attacco

Obiettivo: Dimostrare che il sistema DNS rifiuta attivamente i tentativi di aggiornamento dinamico (Dynamic Update) non autenticati provenienti dalla rete esterna, neutralizzando la vulnerabilità sfruttata in precedenza.

Per verificare la tenuta del perimetro di sicurezza, è stato eseguito un test di regressione replicando l'attacco di *Dynamic DNS Update Spoofing* illustrato nel Capitolo 3. Dal nodo `kali_attacker` (situato nella `frontend_net`), è stato inviato un comando `nsupdate` volto a dirottare il record `www.progetto.ns` verso un IP malevolo, omettendo deliberatamente qualsiasi chiave crittografica.

- **Comando:** `docker exec -it kali_attacker bash -c "echo -e 'server 172.20.0.53\nzone progetto.ns\nupdate delete www.progetto.ns A\nupdate add www.progetto.ns 60 A 172.20.0.80\nsend' | nsupdate"`

A differenza di quanto osservato prima dell'implementazione delle policy di sicurezza, il server ha bloccato immediatamente la transazione. Come si evince dall'output del terminale, l'operazione è fallita restituendo il codice di errore `REFUSED`.

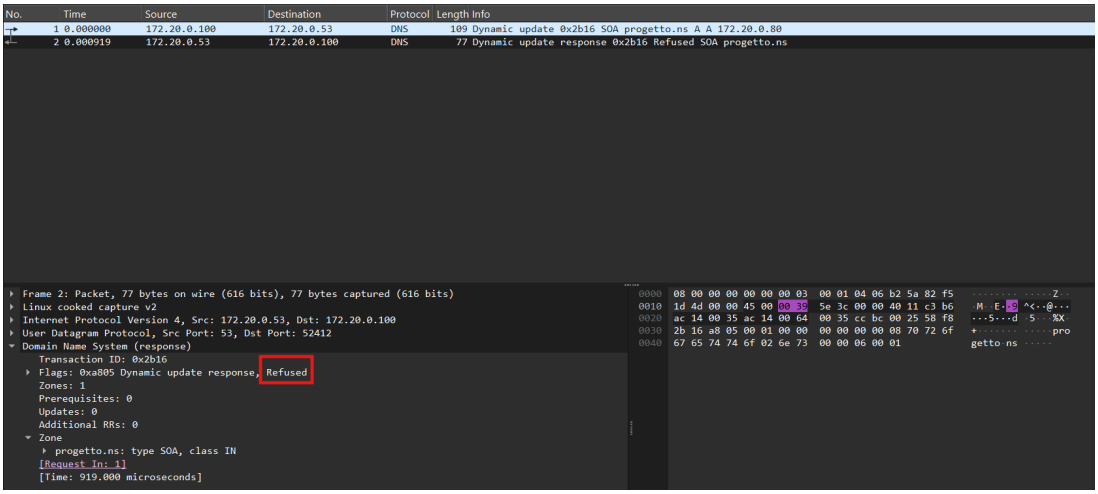
```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it kali_attacker
 bash -c "echo -e 'server 172.20.0.53\nzone progetto.ns\nupdate delete www.p
rogetto.ns A\nupdate add www.progetto.ns 60 A 172.20.0.80\nsend' | nsupdate"

update failed: REFUSED
```

Analisi Forense del Traffico tramite Wireshark

Per comprendere nel dettaglio la risposta del sistema a livello protocollare, il traffico di rete è stato preventivamente messo in ascolto sull'interfaccia del Resolver tramite l'utility `tcpdump`.

- **Comando:** `docker exec bind9_resolver tcpdump -i any port 53 -w /var/log/named/attack_refused.pcap`



L'analisi del file `.pcap` risultante tramite Wireshark ha permesso di esaminare la transazione (Transaction ID: `0x2b16`). Dallo studio dei pacchetti emergono due fattori determinanti:

1. **Assenza di Autenticazione (Query):** Nel pacchetto inviato dal client (172.20.0.100), la sezione "Additional Records" risulta vuota, confermando l'assenza del record TSIG necessario per autorizzare la modifica.
2. **Risposta del Server (RCODE):** Il Resolver (172.20.0.53), agendo da frontiera per l'infrastruttura, rileva l'anomalia e scarta la richiesta ancor prima di coinvolgere il Master Server. Il pacchetto di risposta (visibile nella figura soprastante) presenta il campo Flags impostato su `Reply code: Refused (5)`.

Questo test conferma che il perimetro è stato correttamente blindato: l'architettura implementata scarta in modo nativo qualsiasi pacchetto di aggiornamento che non presenti una firma crittografica HMAC valida, proteggendo l'integrità dei record di zona da attori malevoli esterni.

6.2 Fase 2: Amministrazione Sicura

Obiettivo: Collaudare il regolare funzionamento delle operazioni di amministrazione interne, dimostrando che un aggiornamento legittimo proveniente dalla rete di gestione e provvisto di autenticazione TSIG viene correttamente elaborato dal Master Server.

Dopo aver verificato l'efficacia del blocco contro gli attacchi esterni, è fondamentale assicurarsi che le stringenti policy di sicurezza non inibiscano l'operatività degli amministratori di sistema. In questo scenario, il nodo `wazuh_manager` viene utilizzato come stazione di controllo autorizzata, operante all'interno della rete segregata `mgmt_net` (10.0.1.0/24).

Per motivi di sicurezza (principio del *Least Privilege*), si è scelto di non memorizzare permanentemente file di chiavi crittografiche sul file system del container di monitoraggio. Pertanto, la chiave TSIG necessaria per autorizzare la transazione viene fornita in modalità "inline" tramite il parametro `-y` dell'utility `nsupdate`. Questo approccio permette di passare direttamente l'algoritmo, il nome della chiave e il segreto codificato in Base64 in un'unica istruzione protetta.

Durante la cattura del traffico sul Master Server (`bind9_server`), è stato eseguito il seguente comando dal manager:

- `docker exec -it wazuh_manager bash -c "echo -e 'server 10.0.1.53\nupdate add admin-record.progetto.ns 86400 A 10.0.1.100\nsend' | nsupdate -y hmac-sha256:tsig-update:CxzmZ0nnbzWXnxGjcCuDycgJ6vIplu2Ammo4rEVSO1k="`

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it wazuh_manager
 bash -c "echo -e 'server 10.0.1.53\nupdate add admin-record.progetto.ns 864
00 A 10.0.1.100\nsend' | nsupdate -y hmac-sha256:tsig-update:CxzmZ0nnbzWXnxG
jcCuDycgJ6vIplu2Ammo4rEVSO1k="

C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>
```

Come si evince dall'output del terminale, l'assenza di messaggi di errore o codici `REFUSED` (seguiti dal ritorno pulito al prompt) conferma che il server ha riconosciuto le credenziali e ha applicato la modifica alla zona.

Analisi Forense del Traffico tramite Wireshark

Per certificare l'avvenuta autenticazione a livello di pacchetto, il file `.pcap` (`secure_update.pcap`) è stato analizzato tramite Wireshark. Osservando la transazione (Transaction ID: `0x8ff0`) tra l'IP del Manager (`10.0.1.10`) e l'IP del Master (`10.0.1.53`), si nota un radicale cambiamento nella struttura del payload rispetto all'attacco precedente.

3	0.003963	10.0.1.10	10.0.1.53	DNS	190 Dynamic update 0x8ff0 SOA progetto.ns A 10.0.1.100 TSIG
4	0.027508	10.0.1.53	10.0.1.10	DNS	161 Dynamic update response 0x8ff0 SOA progetto.ns TSIG

```
Domain Name System (query)
    Transaction ID: 0x8ff0
  ▸ Flags: 0x2800 Dynamic update
    Zones: 1
    Prerequisites: 0
    Updates: 1
    Additional RRs: 1
  ▾ Zone
    ▸ progetto.ns: type SOA, class IN
  ▾ Updates
    ▾ admin-record.progetto.ns: type A, class IN, addr 10.0.1.100
        Name: admin-record.progetto.ns
        Type: A (1) (Host Address)
        Class: IN (0x0001)
        Time to live: 86400 (1 day)
        Data length: 4
        Address: 10.0.1.100
  ▾ Additional records
    ▾ tsig-update: type TSIG, class ANY
        Name: tsig-update
        Type: TSIG (250) (Transaction Signature)
        Class: ANY (0x00ff)
        Time to live: 0 (0 seconds)
        Data length: 61
        Algorithm Name: hmac-sha256
        Time Signed: Feb 24, 2026 06:47:09.000000000 ora solare Europa occidentale
        Fudge: 300
        MAC Size: 32
      ▸ MAC
        Original Id: 36848
        Error: No error (0)
        Other Len: 0
  [Response In: 4]
```

```
▸ Frame 4: Packet, 161 bytes on wire (1288 bits), 161 bytes captured (1288 bits)
▸ Linux cooked capture v2
▸ Internet Protocol Version 4, Src: 10.0.1.53, Dst: 10.0.1.10
▸ User Datagram Protocol, Src Port: 53, Dst Port: 44697
▾ Domain Name System (response)
    Transaction ID: 0x8ff0
  ▸ Flags: 0xa800 Dynamic update response, No error
    Zones: 1
    Prerequisites: 0
    Updates: 0
    Additional RRs: 1
  ▾ Zone
    ▸ progetto.ns: type SOA, class IN
  ▾ Additional records
    ▾ tsig-update: type TSIG, class ANY
        Name: tsig-update
        Type: TSIG (250) (Transaction Signature)
        Class: ANY (0x00ff)
        Time to live: 0 (0 seconds)
        Data length: 61
        Algorithm Name: hmac-sha256
        Time Signed: Feb 24, 2026 06:47:09.000000000 ora solare Europa occidentale
        Fudge: 300
        MAC Size: 32
      ▸ MAC
        Original Id: 36848
        Error: No error (0)
        Other Len: 0
  [Request In: 3]
  [Time: 23.545000 milliseconds]
```

1. **Analisi della Query (Autenticazione del Client):** A differenza del pacchetto "sporco" analizzato nella Fase 1, la query legittima contiene dati cruciali nella sezione "Additional records". Come evidenziato nello screenshot, è presente un record associato alla chiave `tsig-update` di tipo `TSIG`. All'interno di questo record sono visibili l'algoritmo concordato (`hmac-sha256`) e, elemento fondamentale, il campo **MAC (Message Authentication Code)**. Quest'ultimo rappresenta l'impronta crittografica generata dinamicamente unendo l'intero payload del pacchetto e la chiave segreta condivisa, garantendo sia l'autenticità del mittente che l'integrità del dato in transito.
2. **Analisi della Risposta (Autenticazione Mutua):** Il Master Server, ricevendo il pacchetto, ricalcola autonomamente il MAC. Coincidendo con quello inviato, il server accetta la modifica e risponde con il flag `Reply code: No error (0)`. Come visibile nell'elenco generale dei pacchetti, anche la risposta del Master include un record TSIG: questo meccanismo di **autenticazione mutua**

garantisce al Manager Wazuh che la conferma di avvenuto aggiornamento provenga effettivamente dal server autoritativo e non sia frutto di *spoofing*.

6.3 Fase 3: Validazione Crittografica e Risoluzione DNSSEC

Obiettivo: Verificare che l'architettura eroghi correttamente i record di zona corredati dalle rispettive firme crittografiche (RRSIG) e dimostrare la resilienza della risoluzione DNS nei confronti dei client. L'ultimo step del collaudo infrastrutturale riguarda la protezione della risoluzione DNS lato client. Nel Capitolo 3, si è dimostrato come l'attaccante fosse in grado di alterare i record di zona sfruttando misconfigurazioni e aggiornamenti non autenticati (*Zone Poisoning*), compromettendo a monte le risposte fornite ai client. Sebbene la mitigazione primaria contro le modifiche non autorizzate sia l'autenticazione TSIG (collaudata nelle fasi precedenti), l'introduzione dello standard DNSSEC (*Inline-Signing*) sul Master Server aggiunge il livello di difesa definitivo: ogni record viene ora firmato crittograficamente in modo trasparente. Questo garantisce ai client l'integrità assoluta del dato, rendendo l'infrastruttura resiliente non solo al veleno iniettato nel database, ma anche a qualsiasi manipolazione del traffico *in-transit*.

Per validare questa architettura, il test è stato suddiviso in due step sequenziali: la verifica della firma sul Master (*Proof of Signing*) e la verifica della risoluzione dal perimetro (*Proof of Validation*).

6.3.1 Step 1: Proof of Signing (Network: Management)

Inizialmente, è stata interrogata l'interfaccia interna del Master Server (`10.0.1.53`) dal nodo di monitoraggio (`wazuh_manager`) per accertare che il motore di firma stesse operando correttamente sui file di zona. Utilizzando l'utility `dig` con i parametri `+dnssec` e `+multiline` (per una formattazione estesa e leggibile delle chiavi), è stato richiesto il record principale del dominio.

- **Comando:** `docker exec -it wazuh_manager dig @10.0.1.53 progetto.ns A +dnssec +multiline`

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it wazuh_manager dig @10.0.1.53 progetto.ns A +dnssec +multiline

; <<>> DiG 9.18.30-0ubuntu0.20.04.2-Ubuntu <<>> @10.0.1.53 progetto.ns A +dnssec +multiline
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 25898
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 4, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1232
; COOKIE: c7b22f157a8bf47401000000699d4217aee8edcf22574881 (good)
;; QUESTION SECTION:
;progetto.ns.           IN A

;; AUTHORITY SECTION:
progetto.ns.            86400 IN SOA ns1.progetto.ns. admin.progetto.ns. (
                                11         ; serial
                                86400      ; refresh (1 day)
                                86400      ; retry (1 day)
                                86400      ; expire (1 day)
                                86400      ; minimum (1 day)
                                )
progetto.ns.            86400 IN RRSIG SOA 13 2 86400 (
                                20260310054709 20260224044709 30011 progetto.ns.
                                XZbkGV4kOCgFN/yH68Oc2MJYPO4uwp6AndSOC/2U+owS
                                JnGRwUNlmwx6G9xNVDcn1ttfjEHnKBTGOWXQ7IG6mA== )
progetto.ns.            86400 IN NSEC admin.progetto.ns. NS SOA RRSIG NSEC DNSKEY TYPE65534
progetto.ns.            86400 IN RRSIG NSEC 13 2 86400 (
                                20260309195113 20260223192839 30011 progetto.ns.
                                OfnBIz+bEsCvuGw7BG3pgrV+jyjyPd1XcHAvHgnZJFgI
                                ussd9ATklZwfFn7xGtfrim1xvZjs12Sw8myVxA5Vgw== )

;; Query time: 0 msec
;; SERVER: 10.0.1.53#53(10.0.1.53) (UDP)
;; WHEN: Tue Feb 24 07:15:51 CET 2026
;; MSG SIZE  rcvd: 402
```

Come mostrato in figura, sebbene la query richiedesse un record `A` non direttamente assegnato all'apice del dominio, il server ha risposto in modo crittograficamente sicuro restituendo l'autorità della zona. Nell'output è chiaramente visibile come sia il record `SOA` che il record `NSEC` (Next Secure, che prova l'assenza del record A) siano immediatamente seguiti da un blocco **RRSIG** (Resource Record Signature). Questo dimostra matematicamente che la zona è attivamente firmata dalla chiave privata del Master.

6.3.2 Step 2: Proof of Validation (Network: Frontend)

Successivamente, il collaudo è stato spostato sulla rete esterna, interrogando il Resolver perimetrale (`172.20.0.53`) dalla postazione dell'attaccante (`kali_attacker`). Questo test simula un client legittimo che richiede una risoluzione sicura per il servizio web dell'organizzazione (`www.progetto.ns`).

- **Comando:** `docker exec -it kali_attacker dig @172.20.0.53 www.progetto.ns A +dnssec`

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it kali_attacker dig @172.20.0.53 www.progetto.ns A +dnssec

; <<>> DiG 9.20.18-1-Debian <<>> @172.20.0.53 www.progetto.ns A +dnssec
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 62088
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1232
; COOKIE: 0f6b0a3850a974ac01000000699d4313f471c6d09e4ce90d (good)
;; QUESTION SECTION:
;www.progetto.ns.               IN      A

;; ANSWER SECTION:
www.progetto.ns.        86400   IN      A       1.2.3.4
www.progetto.ns.        86400   IN      RRSIG   A 13 3 86400 20260309195113 20260223192839 30011 progetto.ns. DFpINh7+COuyJZgkE69uNLAWQRcQ3DpRhxPtz+UJLjnGo9
Ph5uSstD/B 9LfiEtguDLHGpK2QcKTjtsASHPa+rQ==

;; Query time: 10 msec
;; SERVER: 172.20.0.53#53(172.20.0.53) (UDP)
;; WHEN: Tue Feb 24 06:20:04 UTC 2026
;; MSG SIZE  rcvd: 195
```

L'analisi dell'output certifica il perfetto funzionamento dell'intera catena protocollare attraverso i seguenti indicatori:

1. **Flag DO (DNSSEC OK):** Nella sezione `OPT PSEUDOSECTION`, si evince che il client ha impostato esplicitamente il flag `do`. Questa direttiva segnala al server la capacità del client di elaborare le estensioni di sicurezza, richiedendo l'invio delle firme.
2. **Flag AA (Authoritative Answer):** Nell'header della risposta, il Resolver ha impostato il flag `aa`. Questo è un dettaglio architetturale di grande rilevanza: indica che il Resolver, grazie alla configurazione di *forwarding* e alle *Trust Anchors* pre-condivise, agisce come un proxy autorevole per la zona privata `.ns`, inoltrando l'autorevolezza del Master Server direttamente al client.
3. **Answer Section Sdoppiata:** Nella `ANSWER SECTION`, il Resolver non si limita a restituire l'indirizzo IP richiesto (`1.2.3.4`), ma allega contestualmente il record **RRSIG**. La stringa codificata in Base64 rappresenta la firma digitale univoca del record.

Considerazioni sulla Sicurezza

La presenza sistematica del record `RRSIG` in risposta alle query neutralizza definitivamente le minacce di *Cache Poisoning* e *DNS Spoofing* descritte nei capitoli precedenti.

Qualora un attaccante riuscisse a intercettare il traffico di rete e tentasse di manipolare il payload della risposta (ad esempio, alterando l'indirizzo IP `1.2.3.4` con un IP compromesso), si scontrerebbe con un ostacolo crittografico insormontabile. Non possedendo la chiave privata del Master, l'attaccante non

potrebbe rigenerare una firma RRSIG coerente con il nuovo dato. Di conseguenza, il resolver validante o il client finale, confrontando la chiave pubblica (DNSKEY) con la firma contraffatta, rileverebbero l'incongruenza e scarterebbero la risposta malevola (*Bogus Data*), garantendo un approccio *Zero-Trust* sull'integrità delle risoluzioni DNS.

Analisi Forense del Traffico tramite Wireshark

Per ottenere un riscontro oggettivo a livello protocollare e certificare l'effettivo scambio di parametri crittografici, il traffico generato dalla query di validazione è stato catturato direttamente sull'interfaccia del Resolver. L'acquisizione è avvenuta tramite l'utility `tcpdump`:

- **Comando**: `docker exec -it bind9_resolver tcpdump -i any port 53 -w /var/log/named/dnssec_validation.pcap`

Il file risultante è stato successivamente ispezionato attraverso l'analizzatore di protocollo Wireshark. L'analisi della transazione tra il nodo attaccante (`172.20.0.100`) e il Resolver (`172.20.0.53`) rivela i meccanismi interni dello standard DNSSEC applicati al traffico di rete:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.20.0.100	172.20.0.53	DNS	104	Standard query 0x08a1 A www.progetto.ns OPT
2	0.001004	172.20.0.53	172.20.0.100	DNS	243	Standard query response 0x08a1 A www.progetto.ns A 1.2.3.4 RRSIG OPT

L'ispezione profonda (Deep Packet Inspection) dei singoli frammenti conferma le seguenti dinamiche:

1. **Analisi della Query (Richiesta DNSSEC-Aware):** Nel pacchetto inviato dal client Kali Linux, esplorando la sezione "Additional records" all'interno dello pseudo-record `OPT`, è possibile notare l'impostazione del flag **Z: 0x8000**. Questo parametro, comunemente noto come **DO bit (DNSSEC OK)**, rappresenta la notifica formale con cui il client comunica al server la propria capacità di elaborare e validare i Resource Record di sicurezza. Questo dimostra che il client sta attivamente richiedendo dati crittografati.

```
▸ Frame 1: Packet, 104 bytes on wire (832 bits), 104 bytes captured (832 bits)
▸ Linux cooked capture v2
▸ Internet Protocol Version 4, Src: 172.20.0.100, Dst: 172.20.0.53
▸ User Datagram Protocol, Src Port: 48461, Dst Port: 53
▾ Domain Name System (query)
    Transaction ID: 0x08a1
  ▸ Flags: 0x0120 Standard query
    Questions: 1
    Answer RRs: 0
    Authority RRs: 0
    Additional RRs: 1
  ▸ Queries
  ▾ Additional records
    ▾ <Root>: type OPT
        Name: <Root>
        Type: OPT (41)
        UDP payload size: 1232
        Higher bits in extended RCODE: 0x00
        EDNS0 version: 0
      ▾ Z: 0x8000
          1... .... .... .... = DO bit: Accepts DNSSEC security RRs
          .000 0000 0000 0000 = Reserved: 0x0000
        Data length: 12
      ▸ Option: COOKIE
    [Response In: 2]
```

2. **Analisi della Risposta (Erogazione della Firma):** Nel pacchetto di risposta proveniente dal Resolver, la sezione "Answers" risulta arricchita rispetto a una risoluzione DNS standard. Oltre al record richiesto (`A` con indirizzo `1.2.3.4`), il server allega il record **RRSIG**. Espandendo questo campo, Wireshark permette di visualizzare in chiaro i metadati crittografici: l'algoritmo utilizzato (es. *ECDSA Curve P-256 with SHA-256*), le date di validità temporale della firma e, infine, il campo **Signature**. Questa stringa esadecimale costituisce la firma digitale matematica vera e propria che blinda l'integrità del record.

```
Domain Name System (response)
    Transaction ID: 0x08a1
  ▸ Flags: 0x8580 Standard query response, No error
    Questions: 1
    Answer RRs: 2
    Authority RRs: 0
    Additional RRs: 1
  ▸ Queries
  ▾ Answers
    ▾ www.progetto.ns: type A, class IN, addr 1.2.3.4
        Name: www.progetto.ns
        Type: A (1) (Host Address)
        Class: IN (0x0001)
        Time to live: 86400 (1 day)
        Data length: 4
        Address: 1.2.3.4
    ▾ www.progetto.ns: type RRSIG, class IN
        Name: www.progetto.ns
        Type: RRSIG (46) (Resource Record Signature)
        Class: IN (0x0001)
        Time to live: 86400 (1 day)
        Data length: 95
        Type Covered: A (1) (Host Address)
        Algorithm: ECDSA Curve P-256 with SHA-256 (13)
        Labels: 3
        Original TTL: 86400 (1 day)
        Signature Expiration: Mar  9, 2026 20:51:13.000000000 ora solare Europa occidentale
        Signature Inception: Feb 23, 2026 20:28:39.000000000 ora solare Europa occidentale
        Key Tag: 30011
        Signer's name: progetto.ns
        Signature: 0c5a48361efe08ebb225982413af6e34b016411710dc3a518713edcfe5092e39c6a3d3e1e6e4acb43fc1f4b7e212d82e0cb1c6a4ad9070a4e3b6c0121cf6bead
  ▾ Additional records
    ▾ <Root>: type OPT
        Name: <Root>
        Type: OPT (41)
```

6.4 Fase 4: Analisi delle Segnalazioni (SIEM Monitoring)

Obiettivo: Verificare la visibilità dell'attacco neutralizzato nella Fase 1 all'interno del sistema SIEM (Wazuh) e valutare la capacità di *detection* dell'infrastruttura di monitoraggio post-hardening.

Una volta confermato che il Master Server ha respinto l'attacco a livello di protocollo (restituendo il codice `REFUSED` analizzato tramite Wireshark), è stato condotto un audit sui log centralizzati per verificare se il tentativo di compromissione avesse innescato un allarme di sicurezza sul nodo Manager.

È stata eseguita una ricerca mirata all'interno del file degli alert di Wazuh, cercando le occorrenze legate al fallimento dell'aggiornamento:

- **Comando:** `docker exec -it wazuh_manager grep -i "refused" /var/ossec/logs/alerts/alerts.json`

```
C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>docker exec -it wazuh_manager
 grep -i "refused" /var/ossec/logs/alerts/alerts.json

C:\Users\pasqu\Desktop\Progetto NS\progetto-ns>
```

Esito dell'Analisi:
La ricerca non ha prodotto alcun risultato. Il sistema SIEM non ha generato alcun alert di sicurezza (livello 3 o superiore) in concomitanza dell'attacco bloccato, nonostante l'azione ostile sia stata effettivamente registrata a livello di rete.

Considerazioni Architetturali e Necessità di Tuning (Detection Engineering)

Questa apparente anomalia rappresenta in realtà un caso di studio fondamentale per comprendere i limiti delle soluzioni di sicurezza *out-of-the-box*. Il fallimento nella generazione dell'alert non è imputabile a un malfunzionamento del demone DNS (che ha correttamente neutralizzato la minaccia), ma alle logiche di decodifica predefinite del SIEM.

Si possono trarre le seguenti conclusioni tecniche:

1. **Limiti delle regole predefinite (Default Ruleset):** Wazuh, come la maggior parte degli IDS/SIEM commerciali e open-source, è dotato di un set di regole generiche. Un evento come un aggiornamento DNS rifiutato ("*update failed: REFUSED*") viene spesso classificato dai decoder standard come un semplice log informativo o di debug (livello 1 o 2), non sufficiente a superare la soglia di allarme e ad essere scritto in `alerts.json`.
2. **Disallineamento rispetto al Capitolo 3:** Nel Capitolo 3, per rilevare l'avvelenamento della zona, era stata creata una *Custom Rule* che intercettava la stringa "updating zone" (indicativa di un aggiornamento andato a buon fine). In questo nuovo scenario, l'aggiornamento è stato bloccato a monte, cambiando la firma del log ("update failed") e sfuggendo così alla regola personalizzata precedente.
3. **L'importanza del SIEM Tuning:** Questo test dimostra che la mera installazione di un sistema di monitoraggio non garantisce la visibilità sulle minacce. Per sfruttare appieno le potenzialità di un IDS, è indispensabile una fase continua di *Detection Engineering*: gli analisti SOC devono creare *Decoders* e *Custom Rules* modellate in modo sartoriale sul *Threat Model* dell'infrastruttura.

In sintesi, il collaudo della Fase 4 evidenzia come l'infrastruttura sia **intrinsecamente sicura** grazie all'Hardening crittografico (TSIG e DNSSEC), ma sottolinea come la **visibilità sugli incidenti** richieda un'ingegnerizzazione proattiva delle regole di monitoraggio per tradurre i log applicativi in veri e propri *Security Alerts*.

6.5 Sintesi dei Risultati di Collaudo

L'esecuzione del piano di validazione ha permesso di certificare l'avvenuto passaggio dell'infrastruttura DNS da uno stato di vulnerabilità intrinseca (dimostrata nel Capitolo 3) a un'architettura di tipo *Zero-Trust*, conforme ai moderni standard di sicurezza.

L'analisi incrociata dei log applicativi, del traffico di rete e delle dashboard di monitoraggio ha confermato il raggiungimento dei quattro obiettivi prefissati:

- **Neutralizzazione delle Minacce (Fase 1):** Il blocco sistematico degli aggiornamenti dinamici privi di chiave TSIG ha dimostrato la corretta segregazione logica tra la rete pubblica e il database autoritativo, impedendo l'iniezione di record malevoli (*Zone Poisoning*).
- **Operatività Sicura (Fase 2):** L'infrastruttura ha mantenuto la flessibilità necessaria per l'amministrazione di sistema. L'uso di crittografia HMAC per l'autenticazione mutua ha garantito che le operazioni legittime provenienti dal piano di management venissero elaborate senza interruzioni.
- **Integrità della Risoluzione (Fase 3):** L'implementazione di DNSSEC ha spostato il perimetro di sicurezza direttamente sul client. La firma crittografica dei record tramite RRSIG ha reso matematicamente impossibile la manipolazione *in-transit* delle risposte, eliminando alla radice il rischio di dirottamento del traffico legittimo.
- **Visibilità e Detection (Fase 4):** L'audit sulle segnalazioni del SIEM ha evidenziato come l'Hardening infrastrutturale debba essere costantemente affiancato da un'attività di *Detection Engineering*. L'assenza di alert predefiniti a fronte di un attacco correttamente bloccato ha dimostrato l'assoluta necessità di regole customizzate (tuning) per garantire la piena osservabilità degli incidenti di sicurezza.

In sintesi, il collaudo ha evidenziato come l'adozione congiunta di TSIG e DNSSEC non si limiti a mitigare singole vulnerabilità, ma trasformi l'intero paradigma di comunicazione del server, subordinando ogni transazione a una rigorosa validazione crittografica. Al contempo, si è dimostrato che la sicurezza di un'architettura è un processo olistico: la protezione attiva del dato deve sempre evolvere di pari passo con le logiche di monitoraggio centralizzato per assicurare una vera e propria resilienza aziendale.

Nota Architetturale: DNSSEC e le Isole di Fiducia (Island of Trust)

Durante la progettazione e il collaudo dell'architettura DNSSEC, è emersa una sfida infrastrutturale tipica degli ambienti segregati. Nel DNS pubblico (Internet), la distribuzione delle chiavi pubbliche avviene in modo gerarchico attraverso la *Chain of Trust* (Catena di Fiducia): il client non deve conoscere a priori la chiave di un dominio, poiché questa viene garantita dal server genitore (es. il `.it`) tramite un record *DS (Delegation Signer)*, il quale a sua volta è garantito dalla *Root Zone* (`.`).

Poiché l'infrastruttura di test di questo progetto opera su un dominio di primo livello locale e privato (`.ns`), non è stato possibile delegare l'autenticazione a un server genitore. L'architettura è stata pertanto configurata come un'**Island of Trust** (Isola di Fiducia). In questo scenario, le chiavi pubbliche del Master Server (le *Trust Anchors*) sono state distribuite e configurate manualmente (Out-of-Band) all'interno del Resolver validante. Questo approccio non rappresenta una limitazione didattica, bensì riflette le reali e rigorose pratiche di *Hardening* crittografico adottate in reti aziendali isolate, intranet bancarie chiuse o contesti militari (*Air-Gapped Networks*).

Capitolo 7: Conclusioni e Sviluppi Futuri

7.1 Sintesi del Progetto e Obiettivi Raggiunti

Il presente elaborato progettuale si è posto l'obiettivo di analizzare e mitigare le vulnerabilità intrinseche del protocollo DNS, dimostrando come un'architettura inizialmente basata sulla fiducia implicita possa essere compromessa e, successivamente, evoluta in un ecosistema *Zero-Trust* resiliente.

Attraverso la simulazione in un ambiente containerizzato, è stato possibile tracciare l'intero ciclo di vita di un attacco: dalla fase di ricognizione fino all'avvelenamento della cache (Cache Poisoning) tramite l'abuso dei *Dynamic Updates*. La successiva fase di *Hardening* ha dimostrato l'efficacia di un approccio di *Defense-in-Depth* (Difesa in Profondità). L'implementazione del protocollo **TSIG** ha blindato le comunicazioni server-to-server e le operazioni amministrative, mentre l'integrazione di **DNSSEC** (*Fully Automated*) ha esteso il perimetro di sicurezza fino ai client finali, garantendo l'integrità crittografica e l'autenticità di ogni singola risoluzione.

Parallelamente, l'infrastruttura di monitoraggio basata sull'HIDS **Wazuh** ha fornito la visibilità necessaria per tracciare gli eventi critici, dimostrando che la protezione del dato (Prevention) deve sempre essere affiancata da una solida capacità di rilevamento (Detection).

7.2 Limiti dell'Ambiente e Lezioni Apprese (Lessons Learned)

Oltre al successo nell'implementazione delle difese crittografiche, lo sviluppo del progetto ha fatto emergere due importanti lezioni tecniche e metodologiche:

1. **L'astrazione di Rete nei Sistemi Containerizzati:**
   Durante la fase offensiva (Capitolo 3), si è riscontrato che i tentativi di *Blind Spoofing* diretto tramite strumenti come Scapy venivano vanificati non dal protocollo DNS, ma dai meccanismi di Source NAT (SNAT) imposti dal bridge di Docker e dall'hypervisor sottostante. Questa limitazione empirica dimostra che il *Threat Modeling* deve sempre tenere conto del piano di rete reale su cui operano i servizi: in ambienti cloud-native o fortemente virtualizzati, alcune classi di attacco classiche a livello di rete (L2/L3) risultano impraticabili, costringendo l'attaccante – e di conseguenza il difensore – a spostare il focus sulle vulnerabilità logiche e applicative (L7).
2. **L'illusione del SIEM "Out-of-the-Box":**
   Come evidenziato nell'ultima fase di collaudo, un sistema di monitoraggio appena installato è spesso cieco di fronte a log applicativi specifici. Il blocco di un attacco non ha generato allarmi predefiniti, evidenziando che l'acquisizione dei log non equivale alla loro comprensione. Un'infrastruttura sicura richiede una pratica continua di *Detection Engineering*, in cui gli analisti scrivono e raffinano *Custom Rules* modellate in modo sartoriale sull'infrastruttura da proteggere.

7.3 Sviluppi Futuri

L'architettura realizzata costituisce una solida base operativa, facilmente scalabile ed estendibile. Tra i possibili sviluppi futuri per incrementare ulteriormente il livello di maturità dell'infrastruttura si individuano:

- **Integrazione di un NIDS (Network Intrusion Detection System):**
  Attualmente il monitoraggio è affidato a un HIDS (Wazuh) in modalità *agentless*, che analizza i log applicativi. Affiancare un NIDS come *Suricata* o *Zeek*, posizionato sul bridge di rete della *Frontend Network*, permetterebbe di analizzare il payload dei pacchetti in transito, rilevando firme di attacco note (come tentativi di *Zone Transfer* malevoli o traffico anomalo) ancor prima che questi raggiungano il demone DNS.
- **Threat Intelligence e Risposta Automatica (Active Response):**
  Il SIEM Wazuh potrebbe essere integrato con feed di *Threat Intelligence* (es, VirusTotal) per arricchire gli alert. Inoltre, si potrebbero implementare script di *Active Response* che, a fronte di ripetuti tentativi di *Dynamic Update* non autorizzati o query anomale, istruiscano automaticamente il firewall di frontiera (es. `iptables` o le policy di rete Docker) per bannare temporaneamente l'indirizzo IP sorgente dell'attaccante.
- **Gestione Centralizzata delle Chiavi (KMS):**
  Sebbene l'approccio *Island of Trust* sia perfetto per reti isolate, in uno scenario ibrido orientato al Cloud le *Trust Anchors* e le chiavi TSIG potrebbero essere gestite tramite un *Key Management Service* (es. HashiCorp Vault). Questo permetterebbe la rotazione automatizzata e sicura dei segreti senza alcun intervento manuale sui file di configurazione dei container.